

Technologies et langages pour le Web

Sécurité et performance

Z548CU17

Yannick Loiseau

Université Clermont-Auvergne

Master GLIA

version du 17 septembre 2025

Yannick Loiseau

- ▶ `yannick.loiseau@uca.fr`
- ▶ bureau D008 (bâtiment ISIMA)

Évaluation

- ▶ contrôle continu :
 - ▶ écrit : questions de cours (50%)
 - ▶ mini-projet : code et rapport (45%)
 - ▶ séances de TP (5%)

Qu'est-ce que le Web ?

Web $\neq$ Net

~~site internet~~

Internet

Réseau de réseaux $\Rightarrow$ interconnexion de machines

Buts

- ▶ tolérance aux pannes
- ▶ abstraction de l'infrastructure physique
- ▶ « intelligence » à la périphérie (neutralité)

Neutralité

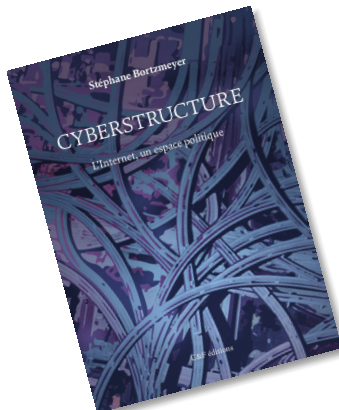
- réseau transparent
- non discriminant
- transporte tout paquet indépendamment :
 - de la source
 - de la cible
 - du contenu

- dégradation (concurrents)
- paiement de la source (double financement)
- paiement du client pour certains services
- blocage/filtrage de sites/services (légaux)

aspects techniques (QoS)

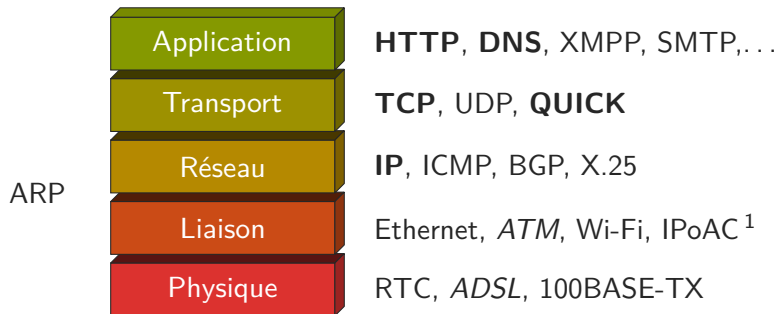
Cyberstructure : L'Internet, un espace politique (2018)

BORTZMEYER



⇒ protocoles en couches

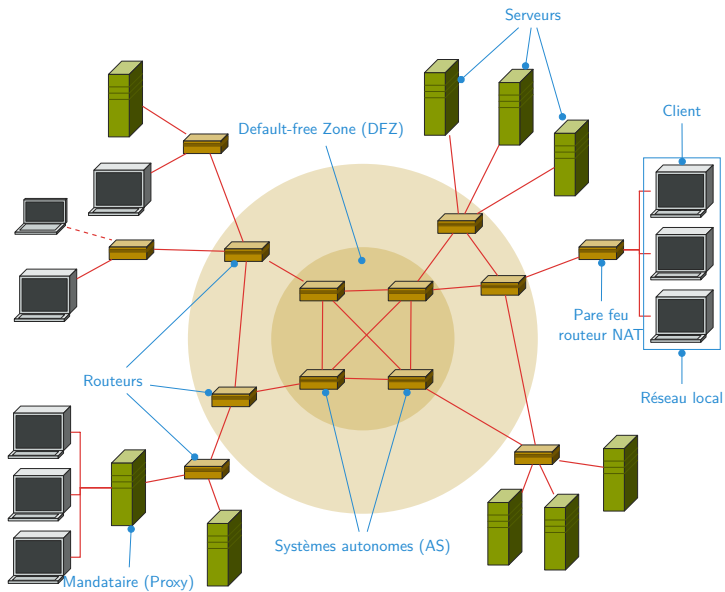
- ▶ isolation
- ▶ indépendance des niveaux → évolution
- ▶ ⇒ connaissance limitée du système global



1. RFC 1149 ; 2001 : 9 paquets 5km, 55% perte, latence 1h

Exemple (Services)

- ▶ mail (SMTP, IMAP, POP3)
- ▶ chat/voip (IRC, XMPP, SIP)
- ▶ streaming (RTP)
- ▶ sync. de temps (NTP)
- ▶ transfert de fichiers (FTP, Bittorent)



commande rigolote

tracert

Univ. → example

```
$ traceroute -A -q 1 www.example.net
1 193.55.95.126 (193.55.95.126) [AS2200] 1.534 ms
2 195.221.123.253 (195.221.123.253) [AS2200] 1.386 ms
3 *
4 *
5 te0-0-0-2-lyon1-rtr-001.noc.renater.fr (193.51.189.169) [AS2200] 11.399 ms
6 *
7 xe-4-3-0-100.mrs10.ip4.tinet.net (77.67.90.117) [AS3257] 20.176 ms
8 xe-0-0-0.mil21.ip4.tinet.net (89.149.184.113) [AS3257] 23.706 ms
9 mno-b2-link.telial.net (80.239.128.117) [AS1299] 23.456 ms
10 prs-bb1-link.telial.net (213.155.136.180) [AS1299] 23.435 ms
11 ash-bb4-link.telial.net (80.91.252.49) [AS1299] 103.151 ms
12 ash-b1-link.telial.net (80.91.245.66) [AS1299] 187.570 ms
13 edgecast-ic-300286-ash-bb1.c.telial.net (62.115.12.62) [AS1299] 108.449 ms
14 93.184.216.119 (93.184.216.119) [AS15133] 103.128 ms
```

commandes rigolotes

- ▶ `whois`
- ▶ `geoiplookup`

Univ. → example

```
$ traceroute -q 1 www.example.net | sed '1d' | egrep -o  
'[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' | xargs -i geoiplookup {}
```

City : FR, 98, Auvergne, Aubiere

ASNum : AS2200 Réseau National de telecommunications pour la Technologie

City : FR, A4, Champagne-Ardenne, Blaise

ASNum : AS2200 Réseau National de telecommunications pour la Technologie

City : FR, A8, Ile-de-France, Maisons-alfort

ASNum : AS2200 Réseau National de telecommunications pour la Technologie

City : DE, 01, Baden-Wurttemberg, Isenburg

ASNum : AS3257 Tinet SpA

City : DE, N/A, N/A, N/A

ASNum : AS3257 Tinet SpA

City : EU, N/A, N/A, N/A

ASNum : AS1299 TeliaSonera International Carrier

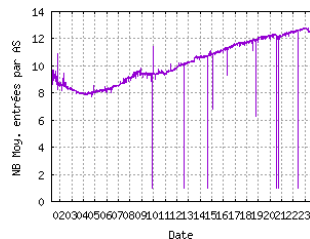
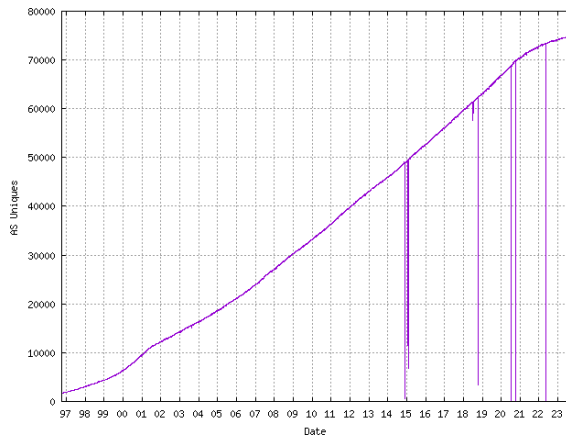
City : US, N/A, N/A, N/A

ASNum : AS15133 EdgeCast Networks, Inc.

AS2200

Internet

Systèmes autonomes (AS)



<http://www.cidr-report.org/>



- ▶ IETF (Internet Engineering Task Force – 1986) : standards du net (RFC)
<http://www.ietf.org/>



- ▶ IANA (Internet Assigned Numbers Authority – 1988) : type média, ports de protocoles, codes divers, . . .
<http://www.iana.org/>



- ▶ ICANN (Internet Corporation for Assigned Names and Numbers – 1998) : adresse IP, TLD, DNS racines (AFNIC)
<http://www.icann.org/>



- ▶ ISOC (Internet Society – 1992) : gestion, organisationnel, . . .
<http://www.internetsociety.org/>

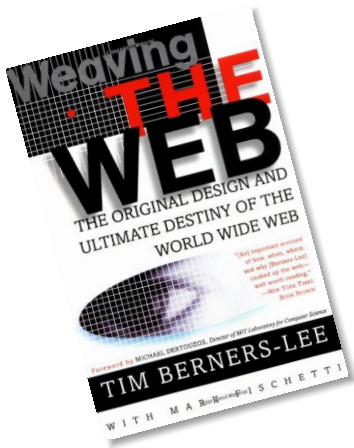
- ▶ ARPA (dpt. de la défense US)
- ▶ fin 60 début 70 :
premiers nœuds ARPANET : 1969 (RFC 1)
- ▶ RFC 675 – Internet transmission control program :
1974
- ▶ IP (Internet Protocol) et TCP (Transmission Control Protocol) (ou UDP – User Datagram Protocol) : RFC 791–RFC 793 (1981)

Suppose all information stored on computers everywhere were linked. Suppose I could program my computer to create a space in which anything could be linked to anything. There would be a single, global information space.

— Tim Berners-Lee (1980)

Weaving the Web (2000)

BERNERS-LEE





Le Web

Interconnexion de ressources $\Rightarrow$ hypermédia

$\Rightarrow$ couche applicative

Buts

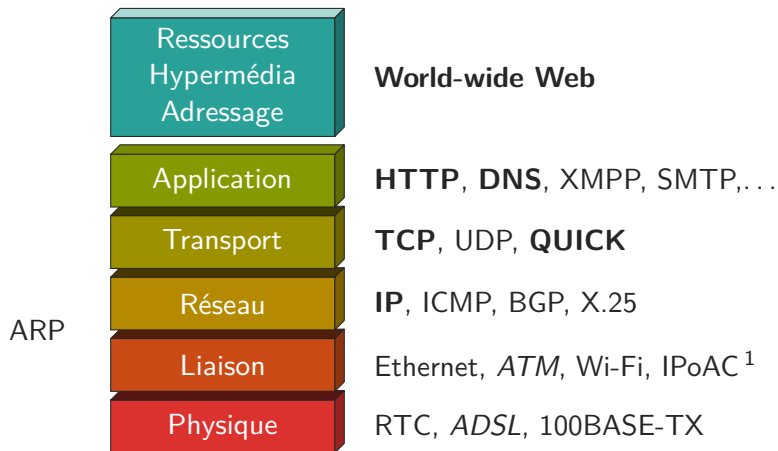
Système d'informations

- ▶ non/semi structurées
- ▶ global
- ▶ à grande échelle
- ▶ décentralisé
- ▶ non supervisé
- ▶ tolérant aux pannes
- ▶ extensible (évolutif)

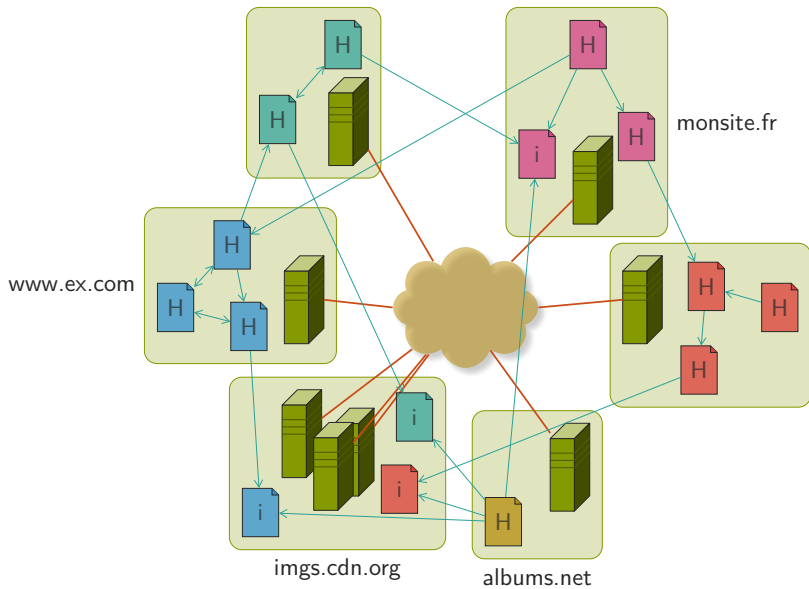
⇒ contraintes architecturales

FIELDING 2000 : « Architectural Styles and the Design of Network-based Software Architectures. »

- ▶ client/serveur
- ▶ faible couplage : client générique, hypermédia
- ▶ ressources
- ▶ représentation homogènes (HTML)
- ▶ protocole : interface uniforme (HTTP)
- ▶ adressage uniforme : indépendance (URL)



1. RFC 1149 ; 2001 : 9 paquets 5km, 55% perte, latence 1h

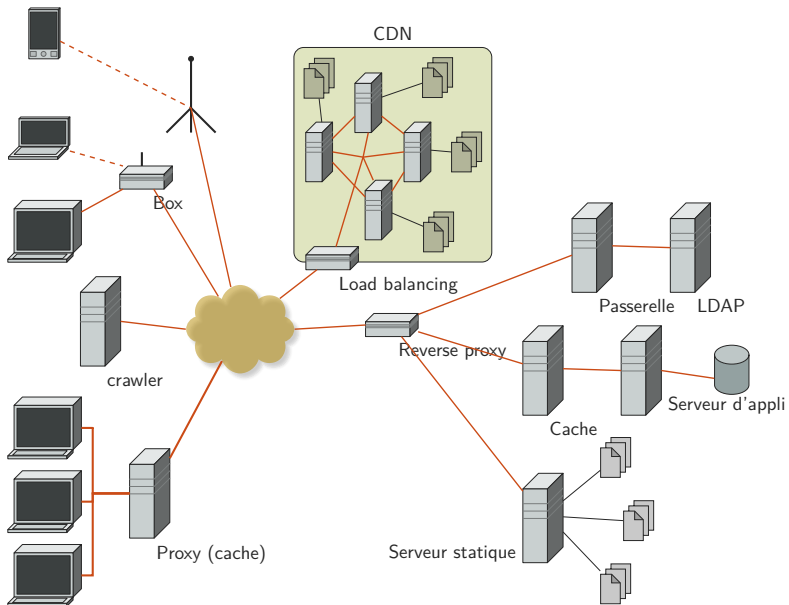


Site web ?

- ▶ ≠ machine
- ▶ ≠ domaine
- ▶ apparence ?
- ▶ ⇒ ensemble de ressources « cohérentes »

⇒ ressources / liens

- ▶ Hypermédia : 1962-65 (D. Engelbart, T. Nelson)
- ▶ T. Berners-Lee (CERN) : ENQUIRE (1980); proposition (1989)
- ▶ HTTP/0.9, HTML, httpd, navigateur (CERN 1990-91)
- ▶ Le CERN « libère » le Web en 1993; W3C (1994)
- ▶ HTTP (1996-99), URI (1998), Apache httpd (1995), libwww(-perl)
- ▶ REST (PhD Th. 2000)



Définition (Agent)

- ▶ client
- ▶ effectue les requêtes
- ▶ manipule les ressources (traitement, affichage)
- ▶ maintient l'état

Exemple

- ▶ navigateur (firefox, chrome, opera, IE)
- ▶ agrégateur (RSS, Podcasts)
- ▶ robots (indexation, extraction)
- ▶ téléchargement (wget, miroirs, ...)
- ▶ applications spécifiques (clients Twitter, Facebook, etc.)

Définition (Serveur)

- ▶ répond aux requêtes du client
- ▶ gère les ressources
 - ▶ état, création, destruction
 - ▶ représentations (génération)
 - ▶ adressage
- ▶ serveur statique / serveur d'application

Exemple

- ▶ Apache, Nginx, IIS, lighttpd, ...
- ▶ mod_php, node.js, tomcat, glassfish, gunicorn, ...

Définition (Passerelle)

- ▶ conversion de protocoles/formats
- ▶ à la fois serveur et client
- ▶ générique

≠ serveur d'appli. → logique métier *ad hoc*

Exemple

`mod_proxy_ftp`, `mod_proxy_ajp`

Définition (Cache)

- ▶ mémorisation des réponses
- ▶ performances
- ▶ tolérances pannes
- ▶ local $\Rightarrow$ privé (navigateur)
- ▶ intermédiaire $\Rightarrow$ partagé (proxy)
- ▶ $\Rightarrow$ mutualisation

Exemple

Varnish, mod_cache

Définition (Proxy)

- ▶ intermédiaire
- ▶ explicite / transparent
- ▶ partage
- ▶ filtrage et contrôle d'accès (parental, pubs, ...)
- ▶ conversion (optim. mobile)
- ▶ cache
- ▶ masquage de source

Exemple

Squid, polipo, mod_proxy_http,

Définition (Reverse Proxy)

- ▶ intermédiaire coté serveur
- ▶ répartition de charge
- ▶ portail / intégration
- ▶ relais

Exemple

HAProxy, Apache (mod_proxy, mod_proxy_html, mod_proxy_balancer),
nginx, Caddy, Traefik, Pound, ...

Définition (CDN)

- ▶ *content delivey network*
- ▶ ensemble de serveurs
- ▶ miroirs
- ▶ répartis géographiquement
- ▶ public / privé
- ▶ $\Rightarrow$ échelle

Exemple

Amazon, Google, Azure, OVH, ...

Définition (Hypermédia)

- ▶ information unitaire : ressource
- ▶ information structurée : liens (typés)
- ▶ non linéaire (graphe)
- ▶ « navigation » non séquentielle (*découverte dynamique*)
- ▶ inclusion (*transclusion*)
- ▶ $\Rightarrow$ référence

Définition (Ressource)

- ▶ élément d'information
- ▶ abstrait $\Rightarrow \neq$ représentations
- ▶ adressable $\Rightarrow$ URI

Exemple

- ▶ profil sur un réseau social
- ▶ article de blog ou d'actu
- ▶ « les conditions météo à Clermont-Ferrand aujourd'hui »

Les URI : identificateur de ressource

URI

Identifiant global unique pour une ressource

- ▶ URN : Uniform Resource Name
- ▶ URL : Uniform Resource Locator

URN

Exemple

- ▶ `urn:ietf:rfc:2141`
- ▶ `urn:isbn:978-0062515872`

URL

(RFC 3986, RFC 1738)

Définition (URL)

Identifiant de ressource

- ▶ unique : espace de nom par le DNS
- ▶ déréléférençable : suffisant pour retrouver le document
- ▶ définie par le serveur : sans gestion centrale
- ▶ opaque : compréhension (sémantique) uniquement par le serveur

URL

Exemple

- ▶ `http://www.example.com/foo/bar`
- ▶ `ftp://file.server.example/some/path/readme.txt`
- ▶ `mailto:user@example.net?subject=Sujet%20du%20message`
- ▶ `file:///home/zaphod/Documents/myfile.pdf`
- ▶ `data:image/png;base64,iVBORw0KGgoAAAAN...`
- ▶ ...

comment (protocole) où (serveur) quoi (ressource locale au serveur)

http ://www.example.com :80 /foo/bar ?a=1&b=1 #ici

► scheme ○

► hostname ○

► port ○

► path ○

► query string ○

► fragment ○

- ▶ uri opaque : identifiant
- ▶ sémantique serveur :
 - ▶ ⇒ navigation / découverte
 - ▶ ⇒ pas de construction (ou le serveur donne les instructions)
 - ▶ ⇒ pas de compréhension par le client (autre que la structure)
 - ▶ ⇒ découplage client/serveur
 - ▶ ⇒ contrôle total du server

Cool URIs don't change

URIs don't change : people change them

`http://www.w3.org/Provider/Style/URI`

`http://www.example.net/index.php?type=article&id=1234`

- ▶ `http://www.example.net/articles/1234`
- ▶ `http://www.example.net/article-1234`
- ▶ `http://www.example.net/a531ae42-84c1-455e-86a3`

Conf. Apache :

```
RewriteEngine On
```

```
RewriteRule ^([a-z]+)/([0-9]+)$ index.php?type=$1&id=$2
```

http://httpd.apache.org/docs/current/mod/mod_rewrite.html

`http://www.example.net/monimage.jpg`

`http://www.example.net/monimage`

Conf. Apache :

Options +Multiviews

http://httpd.apache.org/docs/current/mod/mod_negotiation.html

Principe : Design d'URL

- ▶ réflexion a priori
- ▶ espace d'adressage abstrait $\Leftrightarrow$ représentation physique
- ▶ indépendante des informations variables

chemin, nom, domaine

- ▶ propriétaire (~john/article)
- ▶ techno (.php, /cgi-bin/,...)
- ▶ format (.html, .jpg,...)
- ▶ accès (public, private)
- ▶ titre, catégorie, structure de l'organisation
⇒ redirection URL canonique
- ▶ status (draft, etc.)
 - ✓ latest, today → uri canonique

Exemple

En favoris, scripts, etc. :

<http://weather.net/fr/clermont-ferrand/today>

redirige vers

<http://weather.net/fr/clermont-ferrand/2020-02-02>

si l'on est le 2 février 2020

redirige vers

<http://weather.net/fr/clermont-ferrand/2020-02-03>

le lendemain. . .

⇒ partage

Exemple

<http://doc.app.com/latest>

redirige vers

<http://doc.app.com/v1.2>

(si la version courante est la 1.2)

Propriétés intéressantes

URI $\equiv$ API

- ▶ courte
- ▶ facile à retenir
- ▶ facile à écrire, dicter, lire
- ▶ « bidouillable »

« bidouillable » ⇒ changement « à la main »

Exemple

- ▶ <http://weather.net/fr/clermont-ferrant/2020-02-02>
 - ▶ <http://weather.net/fr/clermont-ferrant/2020-02-01>
 - ▶ <http://weather.net/fr/clermont-ferrant/2020-02-03>
 - ▶ <http://weather.net/fr/paris/2020-02-02>
- ▶ <http://journal.net/articles/1234>
 - ▶ <http://journal.net/articles/1235>
 - ▶ <http://journal.net/articles/>

Pas requis (opaque)

<http://weather.net/5211c671-a93d-4964-ab54-75ef18babf31>,
<http://weather.net/c4d1a05e-f2f4-47b7-a3fe-2cf1be6ea798>

client : découvrir et suivre les liens

⇒ liens typés ([next](#), [previous](#), [up](#), ...)

⚠ Niveau réseau : identification des machines par l'adresse IP

Exemple

- ▶ IPv4 : 93.184.216.34
- ▶ IPv6 : 2606 :2800 :220 :1 :248 :1893 :25c8 :1946

Problème

- ▶ ≈ difficile à retenir
- ▶ ✗ dépendante de l'architecture réseau

DNS

- ▶ **niveau d'indirection**
- ▶ résolution nom de machine $\Leftrightarrow$ adresse IP
- ▶ $\approx$ plus facile à retenir
- ▶ $\checkmark$ indépendance du réseau
- ▶ système hiérarchique
 - ▶ serveurs
 - ▶ structure

Exemple

`www.example.net` $\Leftrightarrow$ `93.184.216.119`

plus facile ?

`8.8.8.8` vs. `gtf0ks.f1x.sj`

- ▶ répartition de charge : 1 nom $\leftrightarrow$ n IP
- ▶ virtual host : n nom $\leftrightarrow$ 1 IP

```
$ host www.stackoverflow.com | grep 'has address' | sort
stackoverflow.com has address 151.101.129.69
stackoverflow.com has address 151.101.1.69
stackoverflow.com has address 151.101.193.69
stackoverflow.com has address 151.101.65.69
```

```
$ host www.stackexchange.com | grep 'has address' | sort
stackexchange.com has address 151.101.129.69
stackexchange.com has address 151.101.1.69
stackexchange.com has address 151.101.193.69
stackexchange.com has address 151.101.65.69
```

```
$ host serverfault.com | grep 'has address' | sort
serverfault.com has address 151.101.129.69
serverfault.com has address 151.101.1.69
serverfault.com has address 151.101.193.69
serverfault.com has address 151.101.65.69
```

Définition (Représentation)

- ▶ données
- ▶ méta-données
- ▶ $\Rightarrow$ état de la ressource

- ▶ ressource : multiples représentation
- ▶ $\neq$ format : *media type*
- ▶ même URL
- ▶ $\Rightarrow$ négociation

type $\Leftrightarrow$ utilisation

- ▶ traitement automatique
- ▶ affichage

Exemple (Blog)

affichage (HTML), impression (PDF) ou agrégation (RSS)

Exemple (Tableau de données)

- ▶ tableur (CSV)
- ▶ affichage (HTML)
- ▶ visualisation (SVG, jpeg, ...)
- ▶ manipulation interactive (JSON)
- ▶ ...

- ▶ RFC 2046
- ▶ IANA
- ▶ <http://www.iana.org/assignments/media-types/>

type/soustype

- ▶ application
 - ▶ xhtml+xml
 - ▶ json
 - ▶ pdf
 - ▶ gzip
 - ▶ octet-stream
- ▶ audio
 - ▶ mpeg
 - ▶ webm
- ▶ example
- ▶ image
 - ▶ png
 - ▶ jpeg
 - ▶ svg+xml
- ▶ message
 - ▶ http
 - ▶ rfc822
- ▶ model
- ▶ multipart
- ▶ text
 - ▶ plain
 - ▶ csv
 - ▶ html
- ▶ video
 - ▶ mpeg
 - ▶ webm

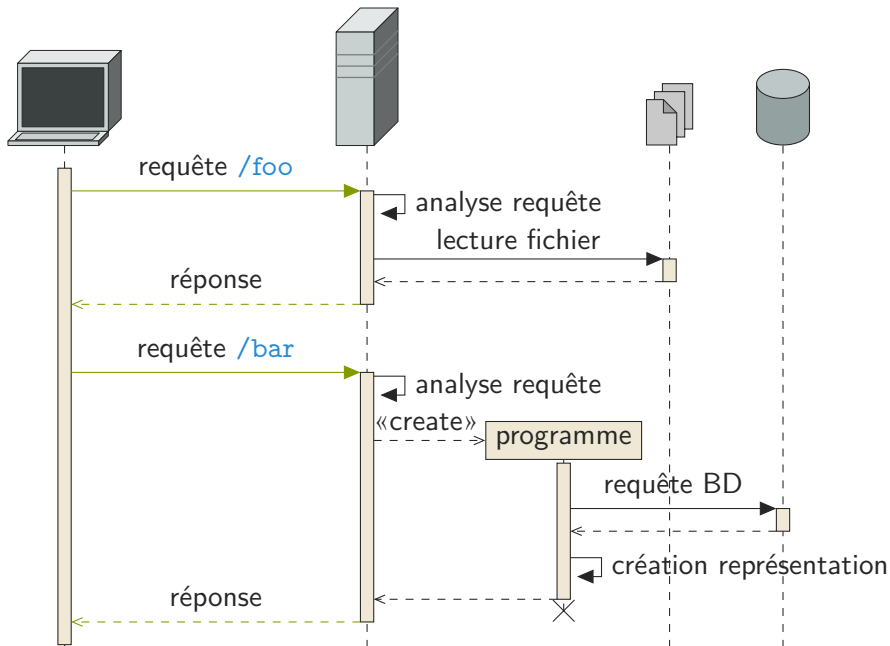
- ▶ $\Rightarrow$ hiérarchique
- ▶ valeur par défaut (`text/*`)
- ▶ gestion par le client $\Rightarrow$ générique/extensible
- ▶ pondération $\rightarrow$ négociation
- ▶ requête / réponse

Protocole HTTP

Hypertext Transfert Protocol :

- ▶ RFC 9110 : sémantique
- ▶ RFC 9112 : HTTP/1.1
- ▶ RFC 9113 : HTTP/2
- ▶ RFC 9114 : HTTP/3
- ▶ https://developer.mozilla.org/en-US/docs/Web/HTTP/Resources_and_specifications

Manipulation des ressources transfert des représentations de l'état



Client/Serveur

- ▶ séparation des responsabilités
- ▶ Données – (Traitements) – Présentation
- ▶ portabilité de l'interface
- ▶ passage à l'échelle du serveur
- ▶ évolution indépendante

Sans état

- ▶ requête suffisante pour répondre
- ▶ pas de contexte client sur le serveur
- ▶ $\Rightarrow$ transparent pour les intermédiaires
- ▶ état de l'application sur le client

Sans état

Avantages

- ▶ visibilité : monitoring → requête (intermédiaires)
- ▶ fiabilité : reprise sur panne plus simple
- ▶ évolutivité (échelle) :
 - ▶ pas d'état $\Rightarrow$ moins de ressource
 - ▶ implém. plus simple
 - ▶ répartition
- ▶ possibilité d'intermédiaires (couches)

Sans état

Inconvénients

- ▶ consistance : pas de contrôle serveur → implém. / comportement clients
- ▶ performances : répétition d'informations

Caches

- ▶ réponse cachable
- ▶ sémantique claire
- ▶ RFC 9111, RFC 5861 (RFC 8246)

Caches

↘ interactions $\Rightarrow$ ↗ performances

- ▶ efficacité
- ▶ échelle
- ▶ latence
- ▶ robustesse

↘ fiabilité

Interface uniforme

- découplage service ↔ implémentation
- évolution indépendante

Contraintes (ReST)

- identification des ressources
- manipulation via représentations
- messages auto-descriptifs
- état conduit par hypermédia (HATEOAS)

Couches

- ▶ système en couches hiérarchiques
- ▶ $\Rightarrow$ intermédiaires
- ▶ visibilité limitée
- ▶ indépendance
- ▶ passage à l'échelle

interface uniforme + couches $\Rightarrow$ *pipe and filter*

composition des composants

flot de données

start line

en-têtes

ligne vide
corps

version, type de message,...
Nom : valeur Nom : valeur ...
...

En-têtes généraux

- ▶ Cache-Control
- ▶ Connection : close, keep-alive
- ▶ Date
- ▶ Pragma
- ▶ Via : intermédiaires
- ▶ ...

Requête

start line : verbe identifiant version

Exemple

```
GET /foo HTTP/1.1
```

Manipulation de ressources

RFC 7231

- ▶ création : **POST**
- ▶ lecture : **GET** (**HEAD**)
- ▶ mise à jour : **PUT** – **PATCH** (RFC 5789)
- ▶ suppression : **DELETE**

Propriétés des méthodes

- idempotence
- sans effet de bord

Définition (Idempotence)

n requêtes $\equiv$ 1 requête

Définition (sans effet de bord)

pas de modification de l'état de la ressource

Propriétés des méthodes

- ▶ indépendance des intermédiaires
- ▶ « cachabilité »
- ▶ reprise sur erreur
- ▶ automatisation / responsabilité

GET /articles/1234?action=delete

Propriétés des méthodes

Méthode	Sans effet de bord	Idempotente
GET	✓	✓
HEAD	✓	✓
PUT	✗	✓
DELETE	✗	✓
POST	✗	✗
PATCH	✗	✗

Propriétés des méthodes

À propos du POST

Pas de sémantique bien définie

- POSTa : append (commentaire, message de forum)
- POSTp : process (traitement quelconque)

RFC 2310 : en-tête **Safe** : (yes|no)

QUERY² : **GET** avec un corps

2. https:

[//www.ietf.org/archive/id/draft-ietf-httpbis-safe-method-w-body-05.html](https://www.ietf.org/archive/id/draft-ietf-httpbis-safe-method-w-body-05.html)

Méta informations, diagnostic et connexion

- ▶ **OPTIONS** : méta-informations
- ▶ **CONNECT** : tunnel
- ▶ **TRACE** : echo

Réponse

start line : version status message

Exemple

HTTP/1.1 200 OK

5 catégories :

- ▶ 1xx : Informations (100 [Continue](#), 101 [Switching Protocols](#))
- ▶ 2xx : Succès (200 [OK](#), 201 [Created](#), 204 [No Content](#))
- ▶ 3xx : Redirection ou pas de contenu (301 [Moved Permanently](#), 304 [Not Modified](#))
- ▶ 4xx : Erreur du client (404 [Not Found](#), 401 [Unauthorized](#), 418 [I'm a teapot](#)³)
- ▶ 5xx : Erreur du serveur (500 [Internal Server Error](#), 504 [Gateway Timeout](#))

<http://www.iana.org/assignments/http-status-codes/>

2xx : Succès

```
GET /example HTTP/1.1
Host : www.example.com
User-Agent : Mozilla/5.0
[...]
```

```
HTTP/1.1 200 OK
Date : Sat, 22 Dec 2012 00 :00 :00 GMT
Server : Apache
Content-Type : text/html
Content-Length : 1270
[...]
```

2xx : Succès

POST /articles/ HTTP/1.1

Host : www.example.com

Content-Type : text/plain

Content-Length : 13

Hello World !

HTTP/1.1 201 Created

Date : Sat, 22 Dec 2012 00 :00 :00 GMT

Location : /articles/42

GET /articles/42 HTTP/1.1

Host : www.example.com

HTTP/1.1 200 OK

Date : Sat, 22 Dec 2012 00 :00 :05 GMT

Content-Type : text/plain

Content-Length : 13

Hello World !

2xx : Succès

POST /articles/42 HTTP/1.1

Host : www.example.com

Content-Type : text/plain

Content-Length : 6

Salut

HTTP/1.1 200 OK

Date : Sat, 22 Dec 2012 00 :00 :20 GMT

Content-Type : text/plain

Content-Length : 19

Hello World !

Salut

2xx : Succès

POST /calculator HTTP/1.1

Host : www.example.com

[donnees a traiter]

HTTP/1.1 202 Accepted

Date : Sat, 22 Dec 2012 00 :00 :00 GMT

Location : /calculator/status/42

2xx : Succès

POST /search HTTP/1.1

Host : www.example.com

[requete complexe (sparql, graphql, ...)]

HTTP/1.1 303 See Other

Date : Sat, 22 Dec 2012 00 :00 :00 GMT

Location : /articles/42

GET /articles/42 HTTP/1.1

HTTP/1.1 200 OK

Date : Sat, 22 Dec 2012 00 :00 :02 GMT

Content-Type : text/plain

Content-Length : 19

Hello World !

Salut

2xx : Succès

DELETE /articles/42 HTTP/1.1

Host : www.example.com

HTTP/1.1 204 No Content

Date : Sat, 22 Dec 2012 00 :01 :00 GMT

GET /articles/42 HTTP/1.1

Host : www.example.com

HTTP/1.1 410 Gone

Date : Sat, 22 Dec 2012 00 :01 :02 GMT

4xx : Erreur du client

```
PATCH /articles/42 HTTP/1.1
```

```
Host : www.example.com
```

```
[...]
```

```
HTTP/1.1 405 Method Not Allowed
```

```
Date : Sat, 22 Dec 2012 00 :01 :00 GMT
```

```
Allow : OPTIONS, GET, HEAD, POST, DELETE
```

```
Content-Length : 0
```

4xx : Erreur du client

```
PUT /articles/42 HTTP/1.1
Host : www.example.com
Content-Type : application/vnd.ms-excel
[...]
```

```
HTTP/1.1 415 Unsupported Media Type
Date : Sat, 22 Dec 2012 00 :01 :00 GMT
Content-Length : 0
```

4xx : Erreur du client

PUT /articles/42 HTTP/1.1

Host : www.example.com

Content-Type : text/plain

[...]

HTTP/1.1 428 Precondition Required

Date : Sat, 22 Dec 2012 00 :01 :00 GMT

Content-Length : 0

- ▶ choix de la représentation
- ▶ dialogue client ↔ serveur
- ▶ RFC 7231, RFC 2295

- ▶ **Accept** : type mime
- ▶ **Accept-Encoding** : compression (**gzip**, ...)
- ▶ **Accept-Language** : langue (**fr**, **en**, ...)
- ▶ **Accept-Charset** : jeux de caractères (**utf-8**, ...)
- ▶ **Negotiate** : **trans**, **vlist**, *

Accept-* : *valeur ;q=poid, valeur...*

Exemple

Accept : text/html, application/xhtml+xml;q=0.8,
application/pdf;q=0.6, text/*;q=0.4, */*;q=0.1

- ▶ **Content-Type** : type mime
- ▶ **Content-Encoding** : compression
- ▶ **Content-Language** : langue
- ▶ **Content-Location** : uri de la représentation choisie

- ▶ **Alternates**
- ▶ **Vary** : negotiate, accept-language, accept-encoding
- ▶ **Link** : </article/a1234.en.html>;rel=alternate;hreflang=en

RFC 8288

- ▶ 300 [Multiple Choices](#)
- ▶ 406 [Not Acceptable](#)

Conf. Apache

Options +Multiviews

- ▶ `articles/a1234.en.html`,
- ▶ `articles/a1234.fr.html`,
- ▶ `articles/a1234.en.html.gz`,
- ▶ `articles/a1234.en.pdf`,
- ▶ ...

```
HEAD /articles/a1234 HTTP/1.1
```

```
Accept : text/html
```

```
Accept-Encoding : gzip
```

```
HTTP/1.1 200 OK
```

```
Content-Type : text/html; charset=iso-8859-1
```

```
Content-Language : en
```

```
Content-Encoding : gzip
```

```
Content-Location : /articles/a1234.en.html
```

```
Vary : negotiate,accept-language
```

```
[...]
```

```
HEAD /articles/a1234 HTTP/1.1
Accept : application/pdf
Accept-Language : fr
```

```
HTTP/1.1 200 OK
Content-Type : application/pdf
Content-Language : fr
Content-Location : /articles/a1234.fr.pdf
Vary : negotiate,accept-language
[...]
```

```
GET /articles/a1234 HTTP/1.1
```

```
Accept : application/json
```

```
HTTP/1.1 406 Not Acceptable
```

```
Alternates : {"/articles/a1234.en.html" 1 {type text/html} {charset utf-8}
```

```
  {language en} {length ...}},
```

```
    {"/articles/a1234.fr.pdf" 0.9 {type application/pdf} {charset utf-8}
```

```
  {language fr} {length ...}}
```

```
...
```

```
Vary : negotiate, accept, accept-language
```

```
[...]
```

```
GET /articles/a1234 HTTP/1.1
```

```
Accept : */*
```

```
Negotiate : trans
```

```
HTTP/1.1 300 Multiple Choices
```

```
Alternates : {"/articles/a1234.en.html" 1 {type text/html} {charset utf-8}  
             {language en} {length ...}},
```

```
             {"/articles/a1234.fr.pdf" 0.9 {type application/pdf} {charset utf-8}  
             {language fr} {length ...}}
```

```
...
```

```
Vary : negotiate, accept, accept-language
```

```
[...]
```

RFC 7232

Identifiant

- ▶ ETag
- ▶ If-Match
- ▶ If-None-Match

Date de modif.

- ▶ Last-Modified
- ▶ If-Modified-Since
- ▶ If-Unmodified-Since

temps serveur $\neq$ temps client

Requête partielle

RFC 7233

- ▶ Range
- ▶ If-Range
- ▶ 206 Partial Content
- ▶ 416 Range Not Satisfiable

- ▶ 304 [Not Modified](#)
- ▶ 412 [Precondition Failed](#)
- ▶ 428 [Precondition Required](#)

HEAD / HTTP/1.1

Host : www.example.com

HTTP/1.0 200 OK

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

HEAD / HTTP/1.1

Host : www.example.com

If-Match : "573c1-254-48c9c87349680"

HTTP/1.0 200 OK

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

HEAD / HTTP/1.1

Host : www.example.com

If-Match : "foobar"

HTTP/1.0 412 Precondition Failed

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

HEAD / HTTP/1.1

Host : www.example.com

If-None-Match : "573c1-254-48c9c87349680"

HTTP/1.0 304 Not Modified

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

```
HEAD / HTTP/1.1
```

```
Host : www.example.com
```

```
If-None-Match : "foobar"
```

```
HTTP/1.0 200 OK
```

```
Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT
```

```
ETag : "573c1-254-48c9c87349680"
```

```
[...]
```

HEAD / HTTP/1.1

Host : www.example.com

If-Modified-Since : Wed, 28 Jul 2010 00 :00 :00 GMT

HTTP/1.0 200 OK

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

HEAD / HTTP/1.1

Host : www.example.com

If-Unmodified-Since : Wed, 28 Jul 2010 00 :00 :00 GMT

HTTP/1.0 412 Precondition Failed

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

HEAD / HTTP/1.1

Host : www.example.com

If-Modified-Since : Sat, 31 Jul 2010 00 :00 :00 GMT

HTTP/1.0 304 Not Modified

Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

ETag : "573c1-254-48c9c87349680"

[...]

HEAD / HTTP/1.1

Host : www.example.com

If-Unmodified-Since : Sat, 31 Jul 2010 00 :00 :00 GMT

HTTP/1.0 200 OK

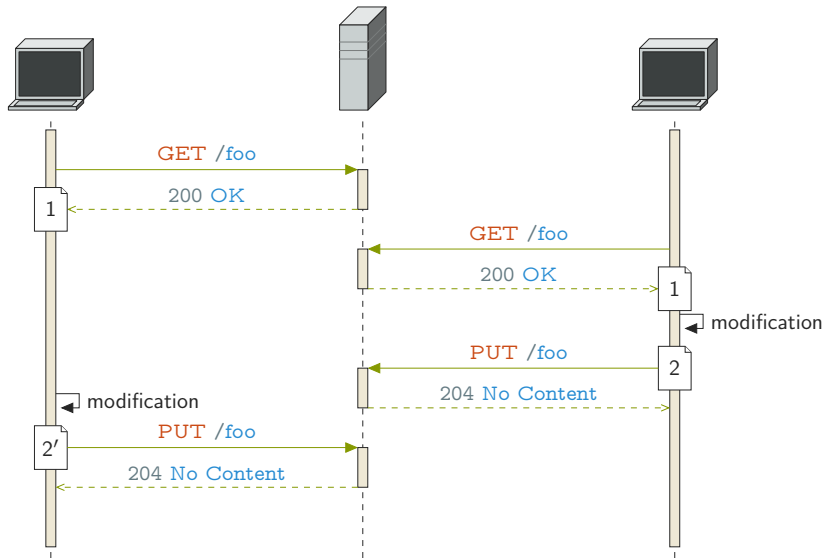
Last-Modified : Fri, 30 Jul 2010 15 :30 :18 GMT

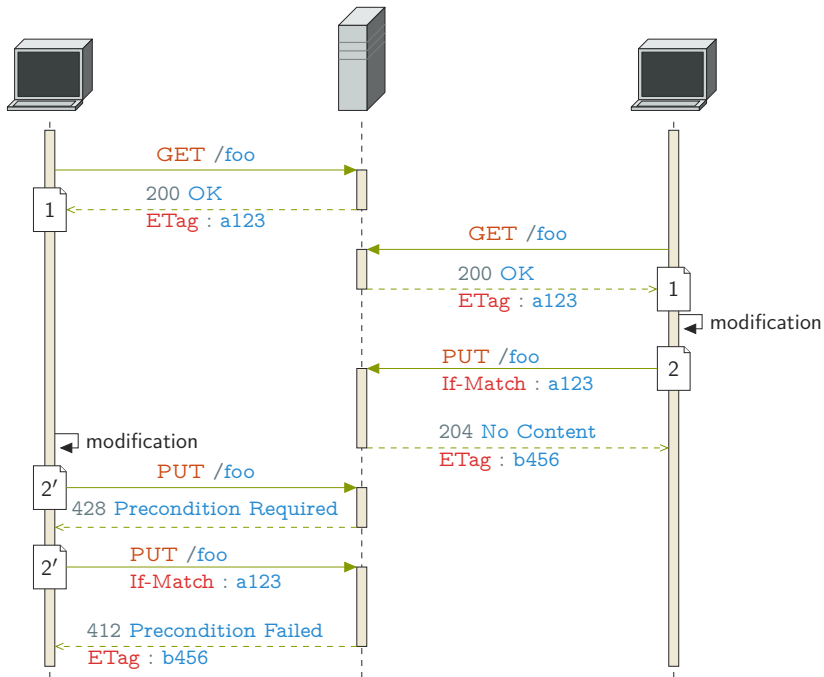
ETag : "573c1-254-48c9c87349680"

[...]

Utilisation

- ▶ rafraichissement du cache
- ▶ modification concurrente





Cache-Control

Cache-Control

Quoi ?

- ▶ `public` (rep)
- ▶ `private` (rep)
- ▶ `no-cache` (req/rep)
- ▶ `no-store` (req/rep)

Cache-Control

Comment ?

- ▶ `max-age` (req/rep)
- ▶ `min-fresh` (req)
- ▶ `max-stale` (req)
- ▶ `s-maxage` (rep)
- ▶ `only-if-cached` (req) → 504 Gateway Timeout
- ▶ `must-revalidate` (rep)
- ▶ `proxy-revalidate` (rep)

Cache-Control

no-transform

- ▶ Safe
- ▶ Via
- ▶ Vary
- ▶ Age
- ▶ ETag
- ▶ Last-Modified
- ▶ Date
- ▶ Expires
- ▶ Warning

- ▶ 203 Non-Authoritative Information
- ▶ 304 Not Modified
- ▶ 504 Gateway Timeout

Principes de base

au delà des fichiers statiques

- ▶ passerelle
- ▶ calcul
- ▶ manipulation de ressources
- ▶ portail
- ▶ ...

⇒ code coté serveur

procédure

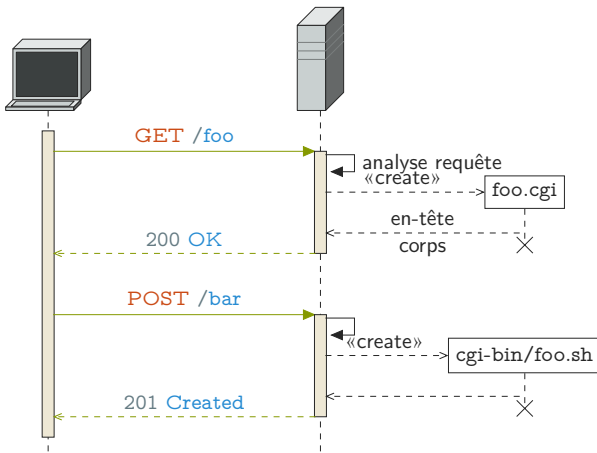
- ▶ entrée : requête
- ▶ sortie : réponse

architecture n-tiers

- ▶ V1.0 1993 NCSA
- ▶ V1.1 1995 → 2004 RFC 3875

programme exécutable

- ▶ autonome
- ▶ n'importe quel langage (script)



configuration serveur $\Rightarrow$ correspondance uri + méthode + en-têtes $\mapsto$ programme

programme

- ▶ test méthode
- ▶ test chemin
- ▶ test en-têtes
- ▶ lecture et analyse du corps
- ▶ génération des en-têtes
- ▶ génération du corps

Variables d'environnement

- ▶ `GATEWAY_INTERFACE` : CGI/version
- ▶ `SERVER_NAME`
- ▶ `SERVER_PROTOCOL`
- ▶ `SERVER_PORT`
- ▶ `REQUEST_METHOD`
- ▶ `PATH_INFO` : chemin relatif au script CGI
- ▶ `SCRIPT_NAME` :
- ▶ `QUERY_STRING` : application/x-www-form-urlencoded brut
- ▶ `REMOTE_ADDR` : IP du client
- ▶ `AUTH_TYPE`
- ▶ `REMOTE_USER`
- ▶ `CONTENT_TYPE` : type média du contenu de la requête
- ▶ `CONTENT_LENGTH`
- ▶ `HTTP_ACCEPT` : type média demandés par le client
- ▶ `HTTP_*` : tous les en-têtes HTTP

GET /cgi-bin/foo.sh/bar/baz?a=1&b=2 HTTP/1.1

SERVER_PROTOCOL : HTTP/1.1

REQUEST_METHOD : GET

PATH_INFO : /bar/baz

SCRIPT_NAME : /cgi-script/foo.sh

QUERY_STRING : a=1&b=2

CONTENT_LENGTH : 0

- ▶ Corps → stdin
- ▶ Réponse → stdout
 - ▶ en-têtes + ligne vide + corps
 - ▶ en-tête spécial [Status](#)

```
#!/bin/sh

if [ $REQUEST_METHOD != "GET" ] ; then
    echo "Content-Type : text/plain"
    echo "Status : 405 NotAllowed"
    echo "Allow : GET"
    echo ""
    echo "Method not Allowed"
    exit 0
fi

echo "Content-Type : plain/text"
echo ""
echo "Hello world!"
```

bibliothèques

- ▶ accès env.
- ▶ parsing *query string* et formulaire (`application/x-www-form-urlencoded`)
- ▶ lecture stdin
- ▶ codes erreurs
- ▶ gestion en-têtes (échapement et encodage des valeurs)
- ▶ génération du corps, échapement html, ...



- ▶ executable « normal » $\Rightarrow \forall$ techno. serveur
- ▶ autonome
- ▶ tests
- ▶ déploiement
- ▶ gestion droits (suexec)
- ▶ isolation

X

- ▶ 1 req. $\Rightarrow$ 1 sous-processus
- ▶ pas d'état partagé :
 - ✓ HTTP (sans état)
 - X** connexion BD, cache interne, ...

CGI → script ⇒ interprété
facile à déployer

interpréteur → serveur

Exemple (Apache)

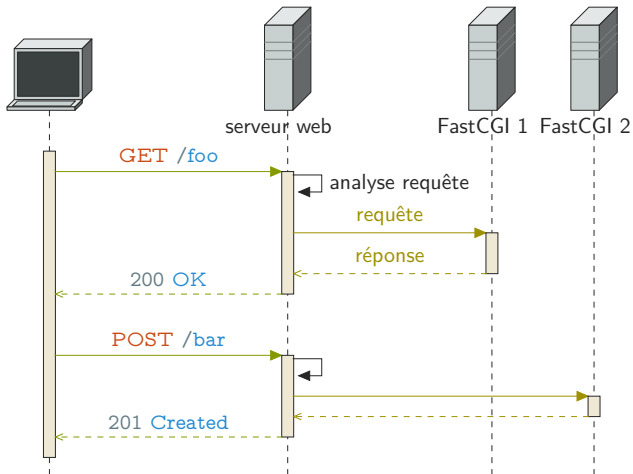
- ▶ mod_perl
- ▶ mod_php
- ▶ mod_python, mod_wsgi
- ▶ mod_lua
- ▶ mod_mono
- ▶ mod_ruby
- ▶ mod_tcl
- ▶ ...

- ▶ $\equiv$ CGI
- ▶ ✓ pas de sous-processus
- ▶ ✓ accès direct aux variables
- ▶ fonctions util.
- ▶ interprété
- ▶ ✗ pas d'isolation

- ▶ 1996 Open Market
- ▶ <http://www.fastcgi.com/devkit/doc/fcgi-spec.html>

Principe

- ▶ $\equiv$ CGI
- ▶ daemon
- ▶ communication $\rightarrow$ socket (protocole spécifique)



Avantages

- ▶ pas de processus $\Rightarrow$ performances
- ▶ isolation $\Rightarrow$ sécurité
- ▶ réutilisation (cache, connexions BD)
- ▶ répartition charge
- ▶ n'importe quel langage et serveur

Outils

- ▶ fcgiwrap
- ▶ spawn-fcgi

Outils

aussi SCGI, AJP

- ▶ conteneur
- ▶ $\cong$ FastCGI
- ▶ $\cong$ embarqué

langage spécifique $\Rightarrow$ accès facilité aux paramètres

- ▶ entrée : objet requête
- ▶ sortie : objet réponse

communication HTTP

Exemple

- ▶ Java : Tomcat, GlassFish, Jetty, WebSphere, WildFly (JBoss)
- ▶ Javascript : Node.js
- ▶ .Net : IIS
- ▶ Python : Gunicorn, Paste, Tornado, uWSGI, Zope
- ▶ Ruby : Passenger, Mongrel, Unicorn

API spécifique

- ▶ Rack (ruby)
- ▶ WSGI (python)
- ▶ servlet, JSP, JAX-(R/W)S (java)
- ▶ ...

≈ lourd

- ▶ $\cong$ serveur d'application
- ▶ léger

- serveur autonome
- embarqué
- reverse proxy
- répartition
- SOA

⇒ micro-services

- ▶ application : sans état
- ▶ maintenu par le client $\Rightarrow$ toutes infos : dans la requête
- ▶ spécifique ?
 - $\Rightarrow$ cookies

Principe

Définition (Cookie)

- ▶ information serveur : clé → valeur + méta
- ▶ spécifiques à l'application
- ▶ opaque pour le client
- ▶ stockée sur le client
- ▶ renvoyée à chaque requête

RFC 6265

Exemple

préférences utilisateur (**Set-Cookie** : **theme=dark**)

Principe

En-têtes

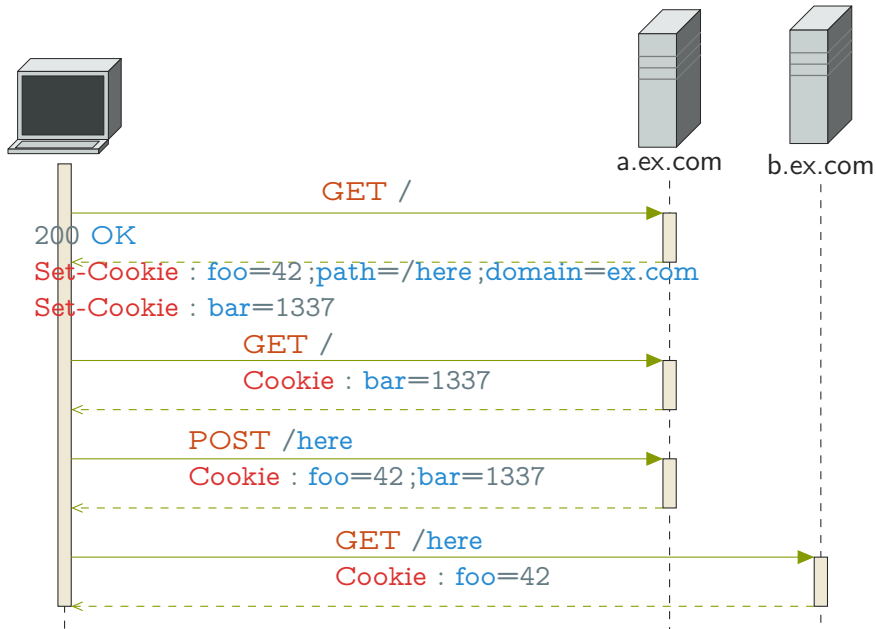
- ▶ **Set-Cookie** dans les réponses
- ▶ **Cookie** dans les requêtes

Principe

Méta-données

champ de validité

- ▶ serveur : [domain](#)
- ▶ chemin : [path](#)
- ▶ durée : [expires](#), [max-age](#)
- ▶ protocole : [secure](#), [httponly](#) (pas API JS)



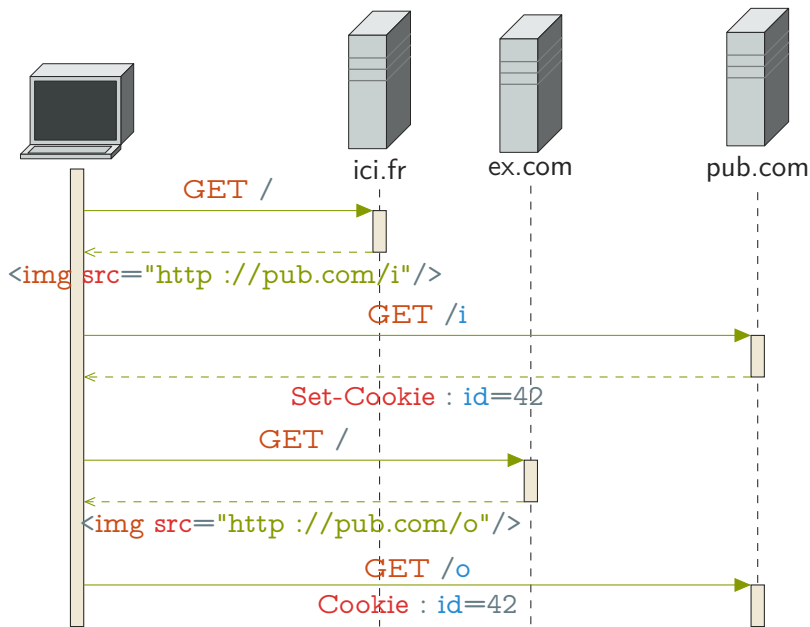
Cookies tiers

Définition (Cookie tier)

cookie dans la réponse d'un serveur tiers (pub, image)



vie privée $\Rightarrow$ blocages, suppression



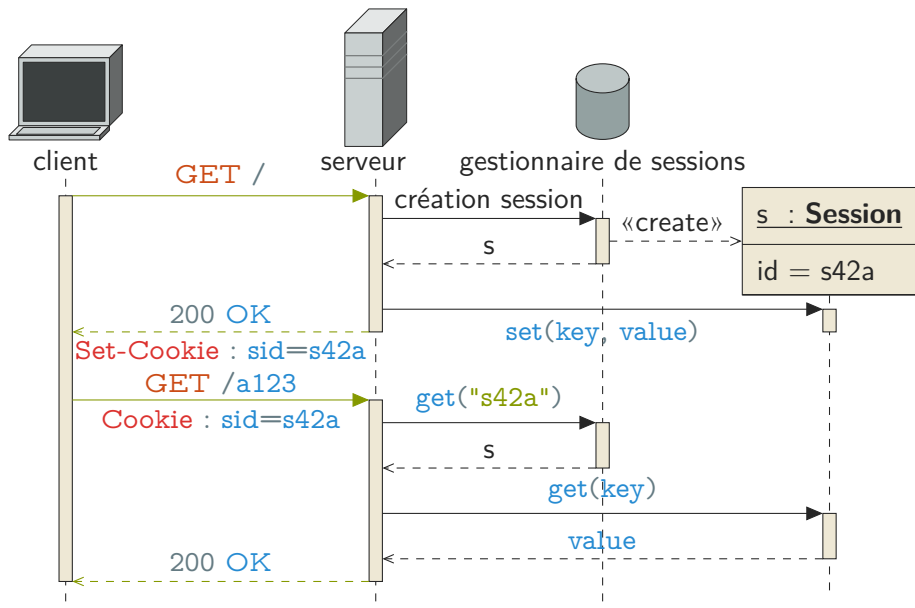
Sessions

Définition

- ▶ structure de donnée
- ▶ mémoire serveur
- ▶ identifiant $\mapsto$ clés/valeurs $\Rightarrow$ cookie

Exemple

authentification, préférences, panier de commande, ...



Sessions

Problèmes

⇒ état coté serveur

- ▶  intermédiaires
- ▶  cache
- ▶  répartition de charge
- ▶  reprise sur panne
- ▶  hypermédia

solution ?

- ▶ persistant entre sessions
- ▶ partagé entre serveurs

≡ ressource non référençable (pas d'url) ⇒ non manipulable

Exemple

commande sur différents terminaux

Sécurité

- ▶ autorité globale (si authentication) : cross-site request forgery (CSRF)
- ▶ données en clair : session hijacking, replay $\Rightarrow$ horodatage + chiffrement + signature
- ▶ faible intégrité : $\neq$ hosts, ports, chemins

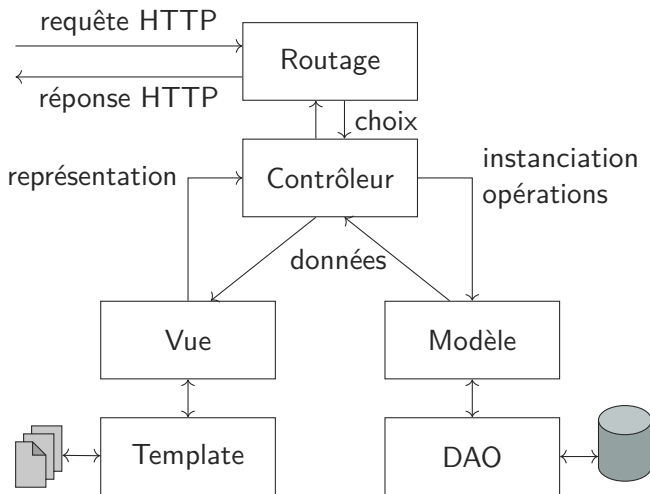
Le modèle MVC

Modèle – Vue – Contrôleur

Modèle : données et règles métier

Vue : présentation et formatage

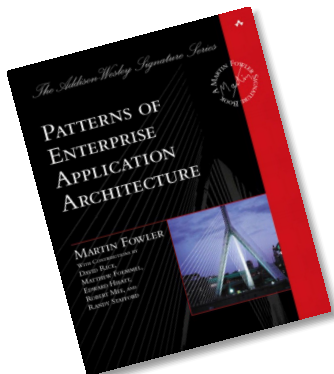
Contrôleur : interactions et connexion M–V



- ▶ Hierarchical MVC
- ▶ Presentation – Abstraction – Control
- ▶ Model – View – Adapter
- ▶ Model – View – Presenter
- ▶ Model – View – View Model (MVVM)

Patterns of Enterprise Application Architecture (2002)

FOWLER



Voir aussi <https://martinfowler.com/eaDev/>

Frameworks

Micro-frameworks

- ▶ Sinatra (Ruby)
- ▶ Flask (Python)
- ▶ Spark (Java)
- ▶ ...

https://en.wikipedia.org/wiki/Category:Web_frameworks

orchestration

- ▶ url (en-têtes) → choix modèle instantiation
- ▶ verbe (en-têtes) → opération
- ▶ extraction information (modèle de vue ?)
- ▶ en-têtes (url) → choix vue
- ▶ génération réponse
- ▶ capture exceptions → code erreur HTTP

⇒ framework

- ▶ code : appels explicite d'API, métadonnées (annotations)
- ▶ configuration
- ▶ convention (nom des classes et paquets)

Jax-RS (Jersey)

```
@Path("hello/{name}")
public class HelloResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String sayHello(@PathParam("name") String name) {
        return String.format("Hello %s!", name);
    }
}
```

Spark

```
public class App {  
    public static void main(String[] args) {  
        get("hello/ :name",  
            (req, res) -> String.format("Hello %s!", req.params(":name")));  
    }  
}
```

- ▶ règles métier
- ▶ accès aux données $\Rightarrow$ persistance

pas de dépendance :

- ▶ contrôleur
- ▶ vue

Définition (Impedance Mismatch)

différence de concepts entre modèles objet et relationnel

- types scalaires vs. références
- (multi)set vs. graphe
- interface uniforme (CRUD) vs. interface spécifique
- déclaratif vs. impératif
- égalité vs. identité
- encapsulation, sous-typage, polymorphisme
- transactions

DAO Data Access Object

⇒ encapsule l'accès aux données

DTO Data Transfert Object
structure, modèle

java.sql.*

```
class UserDAO {  
    public User getUserById(int userId) {  
        PreparedStatement stm = connection.prepareStatement("select name, age  
            from users where id = ?");  
        stm.setInt(1, userId);  
        ResultSet rs = stmt.executeQuery();  
        rs.first();  
        User u = new User(userId);  
        u.setName(rs.getString(1));  
        u.setAge(rs.getInt(2));  
        return u;  
    }  
}
```

org.sql2o.*

```
class UserDao {  
    public User getUserById(int userId) {  
        return connection.createQuery("select id, name, age from users where id =  
            :id")  
            .addParameter("id", userId)  
            .executeAndFetch(User.class);  
    }  
}
```

ORM (*Object-Relational Mapper*)

```
String name = User.getById(1).getName();
```

```
User u = User.getById(42);  
u.setName("Zaphod");  
u.save();
```

« magie noire »

- ▶ configuration (xml)
- ▶ code : annotations, redéfinition, interface
- ▶ convention


```
<hibernate-mapping package="my.application">
  <class name="User" table="users">
    <id name="id" column="id">
      <generator class="increment"/>
    </id>
    <property name="name" column="name"/>
    <property name="age" column="age"/>
  </class>
</hibernate-mapping>
```

javax.persistence.*

```
@Entity
@Table(name="users")
public class User {
    private Integer id;
    private String name;
    private Integer age;

    @Id
    @GeneratedValue(generator="increment")
    @GenericGenerator(name="increment", strategy = "increment")
    public Long getId() {
        return id;
    }

    private void setId(Long id) {
        this.id = id;
    }

    @Column(name="name")
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

- ▶  many to many $\Rightarrow$ jointures
- ▶  synchronisation

Définition (Template)

- ▶ génération de texte
- ▶ patron → forme générale
- ▶ moteur → données

Illimité

langage complet (hôte)

- traitement quelconque
-  peut modifier le modèle

Limité

langage spécifique

- ▶ valeur en lecture
- ▶ pas d'effet de bord

pull : vue (template) → données (modèle)
⇒ à la demande

push : contrôleur (vue) → template
⇒ a priori

- ▶ pipeline
- ▶ callback
- ▶ reverse callback

- ▶ mélange code/template → changement de contexte
- ▶ → appel de code natif
- ▶ *pull*

Exemple

server page (ASP, JSP, PHP)

```
<?php
$data = DAO.getObject($GET["id"]);
?>
<h2><?=$data->getTitle() ?></h2>
<ul>
<?foreach ($data->getElements() as $elt) { ?>
    <li><a href='<?=$elt->url ?>'><?=$elt->name ?></a></li>
<?} ?>
</ul>
```

```
<?php
$data = DAO.getObject($GET["id"]);

echo "<h2>" . $data->getTitle() . "</h2>";
echo "<ul>";
foreach ($data->getElements() as $elt) {
    echo "<li><a href='" . $elt->url . "'>";
    echo $elt->name;
    echo "</a></li>";
}
echo "</ul>";
?>
```

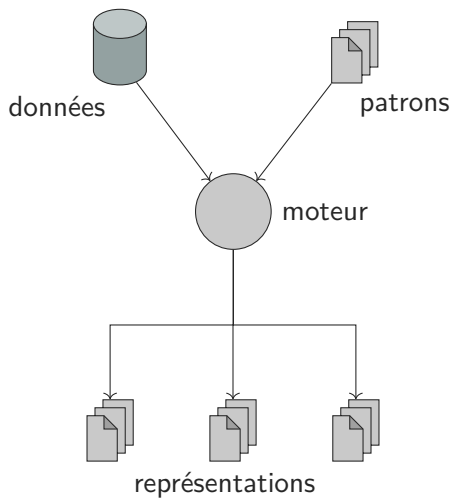
- ✗ mélange logique et présentation
- ≈ affichage
- ✗ $\neq$ représentations $\Rightarrow$ duplication
- ≈  développeur
- ≈  designer

- ▶ patron externe
- ▶ compilé/interprété
- ▶ *push*
- ▶ $\Rightarrow$ langage spécifique

Exemple

- ▶ FreeMarker (Java)
- ▶ Jinja2 (Python)
- ▶ Smarty (PHP)
- ▶ mustache (*)
- ▶ ...

https://en.wikipedia.org/wiki/Template_engine_%28web%29



```
<h2>{{title}}</h2>
{% if elements %}
<ul>
  {%- for name, url in elements %}
  <li><a href="{{url}}">{{name}}</a></li>
  {%- endfor %}
</ul>
{%else%}
<p>Rien...</p>
{%endif%}
```

```
<table><caption>{{title}}</caption>
  <thead><tr><th>Name</th><th>URL</th></tr></thead>
  <tbody>
  {%- for name, url in elements %}
  <tr><td>{{name}}</td>
    <td><a href="{{url}}">{{url}}</a></td></tr>
  {%- endfor %}
  </tbody>
</table>
```

```
from jinja2 import Environment, FileSystemLoader
TEMPLATES, OUT = [Path(p).resolve() for p in sys.argv[1 :3]]
OUT.mkdir(parents=True, exist_ok=True)
env = Environment(loader=FileSystemLoader(TEMPLATES))
data = [
    {
        "title" : "Mon super titre",
        "elements" : [
            ("nom1", "http ://u1.example.net/"),
            ("nom2", "http ://u2.example.net/")
        ],
    },
    {
        "title" : "Autre titre",
        "elements" : []
    }
]

for t in TEMPLATES.glob("*.html") :
    print(f"Render template {t.name}")
    with open(OUT / t.name, 'w', encoding='utf-8') as out :
        for d in data :
            out.write(env.get_template(t.name).render(d))
```

```
<h2>Mon super titre</h2>
```

```
<ul>
```

```
  <li><a href="http ://u1.example.net/">nom1</a></li>
```

```
  <li><a href="http ://u2.example.net/">nom2</a></li>
```

```
</ul>
```

```
<h2>Autre titre</h2>
```

```
<p>Rien...</p>
```

```
<table><caption>Mon super titre</caption>
```

```
  <thead><tr><th>Name</th><th>URL</th></tr></thead>
```

```
  <tbody>
```

```
    <tr><td>nom1</td>
```

```
      <td><a
```

```
        href="http ://u1.example.net/">http ://u1.example.net/</a></td></tr>
```

```
    <tr><td>nom2</td>
```

```
      <td><a
```

```
        href="http ://u2.example.net/">http ://u2.example.net/</a></td></tr>
```

```
  </tbody>
```

```
</table><table><caption>Autre titre</caption>
```

```
  <thead><tr><th>Name</th><th>URL</th></tr></thead>
```

```
  <tbody>
```

```
</tbody>
```


- ✓ sépare logique et présentation
- ✓ affichage
- ≈ duplication de la logique → présentation
- ✓ ✌ développeur
- ✓ ✌ designer

- ▶ logique $\rightarrow$ code
- ▶ formatage élémentaire $\rightarrow$ template
- ▶ $\cong$ D.P. « template method » + `String.format`
- ▶ *push*

- ✓ sépare logique et présentation
- ✗ affichage
- ✓ pas de duplication de la logique
- ✓ 🕊 développeur
- ✗ 💀 designer

```
abstract class ViewModel {  
    abstract String get(String key);  
    abstract Map<String, String> getAll(String key);  
}
```

```
public abstract class DataVue {

    private StringBuilder builder;
    protected DataVue append(String s) {
        builder.append(s);
        return this;
    }

    protected abstract void addTitle(String title);
    protected abstract void addEmptyResult();
    protected abstract void startsElements();
    protected abstract void endsElements();
    protected abstract void addElement(String name, String uri);

    protected void addLink(String content, String url) {
        append("<a href=\"").append(url).append("\">");
        append(content).append("</a>");
    }
}
```

```
public final String render(ViewModel data) {  
    this.builder = new StringBuilder();  
    this.addTitle(data.get("title"));  
    if (data.getAll("elements").isEmpty()) {  
        this.addEmptyResult();  
    } else {  
        this.startsElements();  
        for (Entry<String, String> e : data.getAll("elements").entrySet()) {  
            this.addElement(e.getKey(), e.getValue());  
        }  
        this.endsElements();  
    }  
    return builder.toString();  
}
```

```
public class ListDataVue extends DataVue {  
    protected void addTitle(String title) {  
        append("<h2>").append(title).append("</h2>");  
    }  
    protected void addEmptyResult() {  
        append("<p>Rien</p>");  
    }  
    protected void startsElements() {  
        append("<ul>");  
    }  
    protected void endsElements() {  
        append("</ul>");  
    }  
    protected void addElement(String name, String url) {  
        append("<li>");  
        addLink(name, url);  
        append("</li>");  
    }  
}
```

« Enforcing strict model-view separation in template engines »
(PARR 2004)

- ▶ encapsulation (ordre d'appel, logique métier, valeurs)
- ▶ clarté du code : HTML lisible, affichable
- ▶ séparation des tâches dev. ↔ design.
- ▶ réutilisation : sous-templates, autre modèle
- ▶ maintenance
 - ▶ changement de présentation → pas logique
 - ▶ pas de recompilation
- ▶ sécurité

Définition (Encapsulation du modèle)

toute logique / calculs dépendant des données → modèle

Définition (Encapsulation de la vue)

toutes mise en forme / affichage → vue

Définition (Séparation stricte)

- ▶ M encapsulé
- ▶ V encapsulée
- ▶ pas de types dans V
- ▶ V ne modifie pas M $\Rightarrow$  effets de bord

Règles

1. M : pas d'info. d'affichage
2. V : pas de méth. à effet de bord
3. V : pas de calcul (`<td>TTC</td><td>{{prix * 1.20}}</td>`)
4. V : pas de test métier sur les valeurs `if val < 120` vs. `if valIsOk`
5. V : pas de types complexes (indices, param. de méth.)

Indice : 1–5

⇒ URL → contrôleur (route)

 pull/callback/illimité $\Rightarrow$ non séparé

```
<ul>
<?foreach ($model->getNames() as $name) { ?>
<li><?=$name ?></li>
<?} ?>
</ul>
<p><?=$model->getNumberOfNames() ?></p>
```

pull/limité/pipeline $\Rightarrow$ séparé

Document « à trous » → langage rationnel

⇒ séparation stricte

include/macro $\Rightarrow$ réutilisabilité

+ map / boucles / récursivité → langage hors contexte
⇒ XML

+ test booléen → langage contextuel

⇒ même expressivité illimité

respect de la séparation

Problème

conversion valeurs → chaînes date, valeurs numériques, etc.

- ▶ modèle ? **X** info d'affichage
- ▶ template ? **X** pas de type
- ▶ contrôleur ? **≈**

appel (implicite) à `toString` (ou équivalent)

MVCR : renderer

contrôleur / moteur de template

⇒ i18n


```
def human_date(val) :  
    return ...  
  
def machine_date(val) :  
    return ...  
  
env = jinja2.Environment(...)  
env.filters['hdate'] = human_date  
env.filters['mdate'] = machine_date
```

```
<time datetime="{{begin|mdate}}">{{begin|hdate}}</time>
```

Internationalisation

internet international $\Rightarrow \neq$ langues, cultures

- ▶ symboles
- ▶ représentations
- ▶ codes

$\neq$ représentations et rendus

$\Rightarrow$ négociation + méta-données d'encodage

$\Rightarrow$ négociation + méta-données de langue (RFC 1766)

locale (POSIX)

- ▶ langue
- ▶ sens d'écriture
- ▶ ordre alphabétique
- ▶ formatage des nombre (1,000.5 vs. 1 000,5)
- ▶ formatage de date (2010-10-01, 01/10/10)
- ▶ formatage des monnaies (\$3.5 vs. 3,5 €)
- ▶ règles typographiques (« » vs. "", listes, ident)

i18n → adaptable

- ▶ design + code
- ▶ prise en compte des locales

L10n → adaptation

- ▶ traduction
- ▶ locales
- ▶ styles
- ▶ 1 fois par langue !

- ▶ caractère : abstrait
- ▶ jeu de caractère (charmap) / page de code (code page) :
caractère $\leftrightarrow$ code numérique
- ▶ encodage(charset) : représentation de ce code (nb de bits, ...)
- ▶ fonte : caractère $\leftrightarrow$ glyphe

- ▶  système de fichier (nom)
- ▶  flux (réseau)
- ▶  base de données

toute la chaîne doit soit :

- ▶ être homogène (unicode)
- ▶ gérer l'encodage à chaque niveau $\Rightarrow$ connaître celui des autres

Unicode

- ▶ jeu de caractère
- ▶ encodage (UTF)

+1 000 000

U+000000 → U+10FFFD

encodage

- ▶ UTF-8
- ▶ UTF-16
- ▶ UTF-32

multi-octets

- ▶ petit-boutiste (little endian)
- ▶ gros-boutiste (big endian)

⇒ BOM (byte order mark) : U+FEFF

é : U+00E9

latin1 (ISO 8859-1) : E9

UTF-8 : 2 octets : C3 A9

UTF-16 : 2 octets (plus le BOM) : FF FE | E9 00

UTF-32 : 4 octets (plus le BOM) : FF FE 00 00 | E9 00 00 00

é utf-8 → latin1 « Ã© »

latin1 : C3 → « Ã » A9 → « © »

Ω « U+03A9 GREEK CAPITAL LETTER OMEGA » :

UTF-8 : CE A9

UTF-16 : FF FE | A9 03

UTF-32 : FF FE 00 00 | A9 03 00 00

þ « U+1E55 LATIN SMALL LETTER P WITH ACUTE »

UTF-8 : E1 B9 95 (55 → U)

UTF-16 : FF FE | 55 1E

UTF-32 : FF FE 00 00 | 55 1E 00 00

ℳ « U+1D4B4 MATHEMATICAL SCRIPT CAPITAL Y »

UTF-8 : 4 octets : F0 9D 92 B4

UTF-16 : 2×2 octets + BOM D835 DCB4 →
FF FE | 35 D8 | B4 DC

UTF-32 : 4 octets (plus le BOM) : FF FE 00 00 | B4 D4 01 00

Diacritiques : é

- ▶ U+00E9
- ▶ U+0065 U+0301

Emoji ZWJ : Deaf Woman Medium-Dark Skin Tone

- ▶ U+1F9CF Deaf Person
- ▶ U+1F3FE Medium-Dark Skin Tone
- ▶ U+200D Zero Width Joiner
- ▶ U+2640 Female Sign
- ▶ U+FE0F Variation Selector-16

Problèmes de sécurité

Gestion de l'accès aux ressources

- ▶ identification
- ▶ authentification
- ▶ autorisation

Définition (Identification)

- ▶ associer une identité/un identifiant à l'utilisateur
- ▶ pas forcément un compte
- ▶ pas forcément identité physique

Exemple

- ▶ recherches récentes → recommandations
- ▶ caddie
- ▶ statistiques comportementales
- ▶ corrélation de requêtes
- ▶ **contrôle d'accès**
- ▶ ...

Identification

- ▶ identifiant fourni (**Authorization**, **From**)
- ▶ cookie (uuid)
Set-Cookie :
`user=1f1d86ca-7080-403b-89e9-bc649dd7887e;Expires=Tue, 01-Jan-2036 08:00:01 GMT`
- ▶ certificat
- ▶ implicite
- ▶ ...

Définition (Authentification)

garantir l'identité

Exemple

- ▶ mot de passe (HTTP, sessions)
- ▶ jeton
- ▶ certificat / clé
- ▶ tiers de confiance (OAuth, OpenID, ...)

Définition (Autorisation)

contrôle d'accès : défini ce que l'utilisateur peut faire

⇒ ACL (*access control list*)

groupe, rôle (RBAC)

attributs (ville, service, etc.), règles métier (ABAC), ...

- ▶ applicatif / métier
- ▶ HTTP (frontal)

403 Forbidden

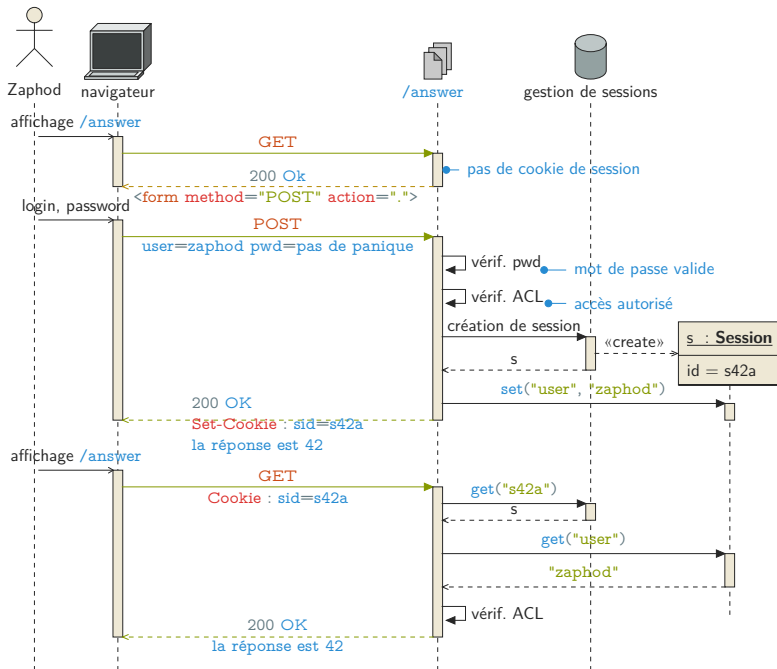
Conf. Apache (fichiers statiques, reverse proxy, ...) :

```
<Location /foo>
  <Limit GET>
    Require valid-user
  </Limit>
  <Limit POST>
    Require group editors
  </Limit>
  <Limit DELETE>
    Require user zaphod trillian
  </Limit>
</Location>
```

<https://httpd.apache.org/docs/2.4/fr/howto/access.html>

- ▶ ad hoc
- ▶ code ressource
- ▶ état coté serveur → session

Version simple



```
GET /answer HTTP/1.1
```

```
Host : h2g2.com
```

```
HTTP/1.1 200 OK
```

```
Content-Type : text/html; charset=UTF-8
```

```
...
```

```
<form action="/answer" method="POST">
```

```
  <input name="u" placeholder="Identifiant" type="text" />
```

```
  <input name="p" placeholder="Mot de passe" type="password"/>
```

```
</form>
```

```
...
```

POST /answer HTTP/1.1

Host : h2g2.com

u=zaphod&p=pas%20de%20panique

HTTP/1.1 200 Ok

Set-Cookie : sid=s42a ; path=/ ; domain=.h2g2.com ; Secure ; HTTPOnly

Content-Type : text/plain

Content-Length : 31

Hello Zaphod, the answer is 42 !

GET /answer HTTP/1.1

Host : h2g2.com

Cookie : sid=s42a

HTTP/1.1 200 Ok

Set-Cookie : sid=s42a ; path=/ ; domain=.h2g2.com ; Secure ; HTTPOnly

Content-Type : text/plain

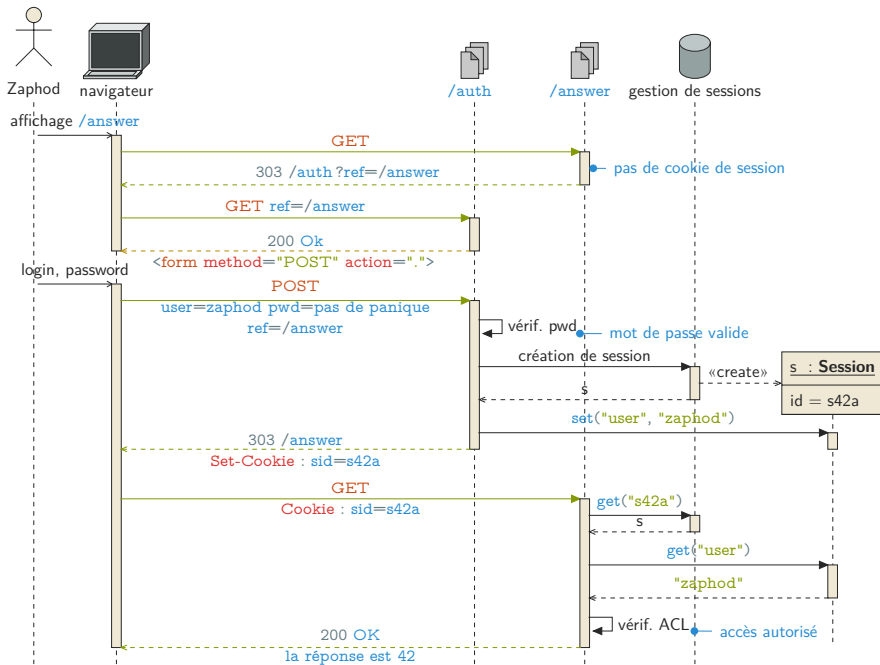
Content-Length : 31

Hello Zaphod, the answer is 42 !

Version simple

- ▶ ✓ interaction simple
- ▶ ≈ gestion pour toutes les ressources
- ▶ ✗ 200 **Ok** mais pas contenu
- ▶ ✗ « gaspille » **POST** (ou surcharge)
- ▶ ✗ ...

Version élaborée



GET /answer HTTP/1.1

Host : h2g2.com

HTTP/1.1 303 See Other

Location : /auth?ref=%2Fanswer

GET /auth?ref=%2Fanswer HTTP/1.1

Host : h2g2.com

```
HTTP/1.1 200 OK
```

```
Content-Type : text/html; charset=UTF-8
```

```
...
```

```
<form action="/auth" method="POST">
```

```
  <input name="u" placeholder="Identifiant" type="text" />
```

```
  <input name="p" placeholder="Mot de passe" type="password"/>
```

```
  <input name="r" type="hidden" value="/answer"
```

```
</form>
```

```
...
```

```
POST /auth HTTP/1.1
```

```
Host : h2g2.com
```

```
u=zaphod&p=pas%20de%20panique&r=%20Fanswer
```

HTTP/1.1 200 OK

Content-Type : text/html ; charset=UTF-8

...

```
<form action="/auth?ref=%2Fanwser" method="POST">
```

```
  <input name="u" placeholder="Identifiant" type="text" />
```

```
  <input name="p" placeholder="Mot de passe" type="password"/>
```

```
</form>
```

...

POST /auth?ref=%2Fanswer HTTP/1.1

Host : h2g2.com

u=zaphod&p=pas%20de%20panique

HTTP/1.1 303 See Other

Location : /answer

Set-Cookie : sid=s42a ; path=/ ; domain=.h2g2.com ; Secure ; HTTPOnly

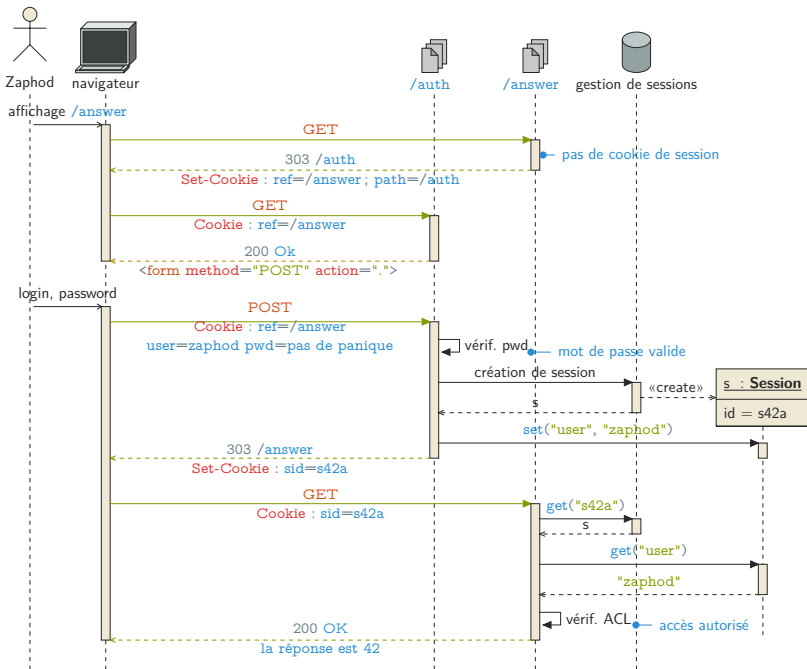
GET /answer HTTP/1.1

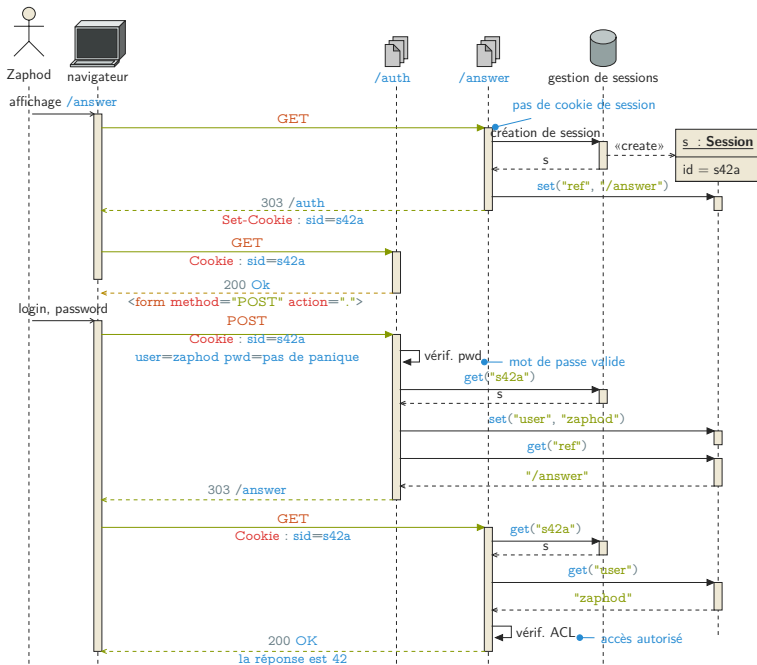
Host : h2g2.com

Cookie : sid=s42a

Variantes

- ▶ origine dans un cookie
- ▶ session immédiate





Formulaire et session

- ▶ ✓ formulaire personnalisé
- ▶ ✓ page oubli / création
- ▶ ≈ authentification persistante
- ▶ ✗ automatisation
- ▶ ✗ cookie
- ▶ ✗ clair $\Rightarrow$ TLS
- ▶ ✗ vol de session (e.g. Wireshark, FireSheep)

- ▶ standard
- ▶ générique
- ▶ sans état
- ▶ HTTP : (rev. prox., app. serv.)

- ▶ **WWW-Authenticate** : type, realm
- ▶ **Authorization** : type, jeton
- ▶ 401 **Unauthorized**

Types

- ▶ Basic : clair (RFC 7617)
- ▶ Digest : chiffré (RFC 7616)
- ▶ Bearer : opaque (RFC 6750)
- ▶ autres...

<http://www.iana.org/assignments/http-authschemes/>

Basic

Jetton : `base64(user + ' ' + password)`

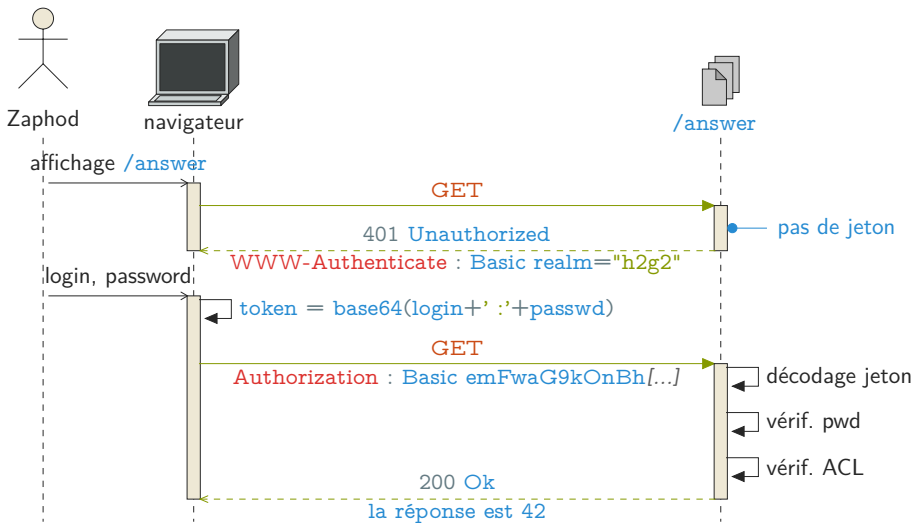
Exemple

WWW-Authenticate : Basic realm="h2g2"

- ▶ user : `zaphod`
- ▶ password : `pas de panique`
- ▶ `base64("zaphod :pas de panique")` →
`emFwaG9kOnBhcyBkZSBwYW5pcXVl`

Authorization : Basic `emFwaG9kOnBhcyBkZSBwYW5pcXVl`

Basic



Digest

- ▶ similaire Basic
- ▶ `nonce`
- ▶ `response` : md5
 - ▶ user
 - ▶ password
 - ▶ method
 - ▶ URI
 - ▶ nonce
 - ▶ compteur, ...

- ▶ ✓ sans état, générique
- ▶ ✓ automatisation (robots, .netrc)
- ▶ ✓ sur un frontal (rev. proxy)
- ▶ ≈ oubli / création → page 401/403 personnalisée
- ▶ ✗ « formulaire » natif

localhost:8080/protected

Authentication requise

 Le site http://localhost:8080 demande un nom d'utilisateur et un mot de passe. Le site indique : « here and now »

Utilisateur :

Mot de passe :

localhost:8080/protected

Ouvrir une session

http://localhost:8080

Nom d'utilisateur

Mot de passe

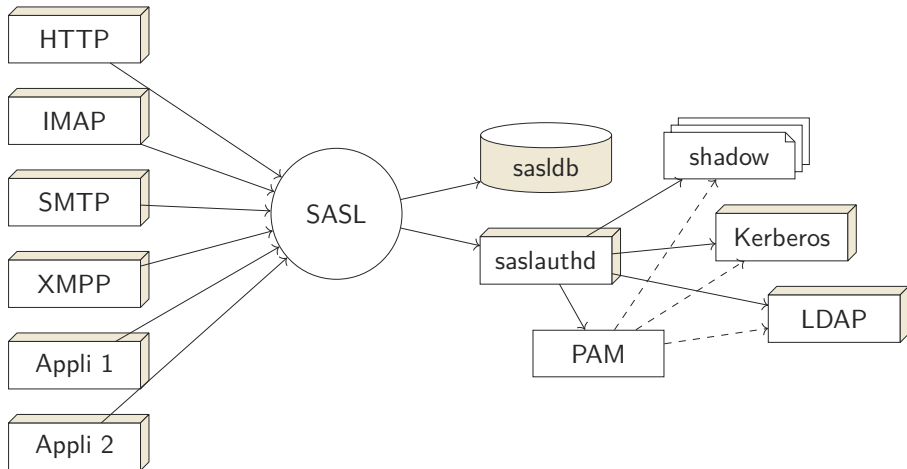
≠ backends

Exemple (apache)

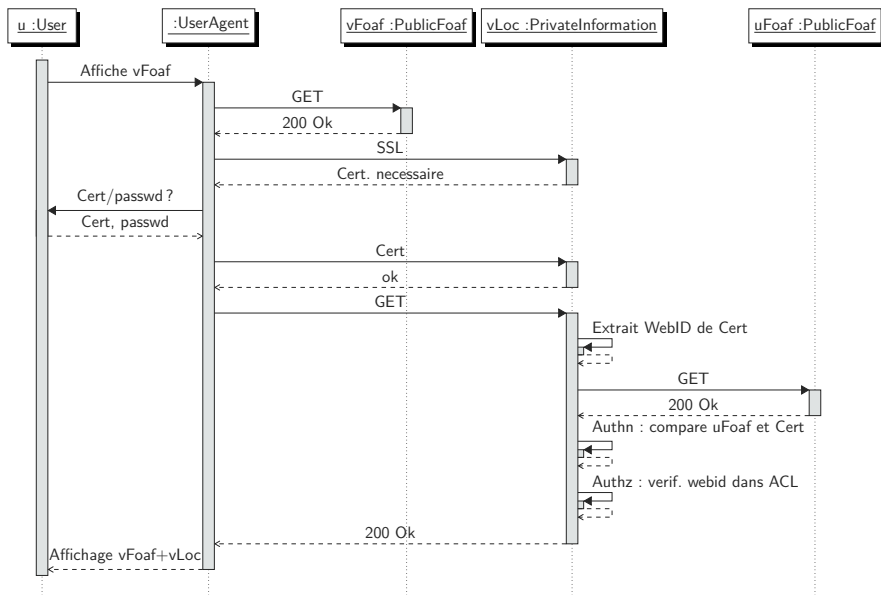
- ▶ htpasswd
- ▶ LDAP
- ▶ NTLM
- ▶ SASL
- ▶ PAM
- ▶ Kerberos
- ▶ SGBD
- ▶ ...

<https://httpd.apache.org/docs/2.4/fr/howto/auth.html>

- ▶ Simple Authentication and Security Layer
- ▶ RFC 4422
- ▶ découplage authentication / application



FoaF + SSL



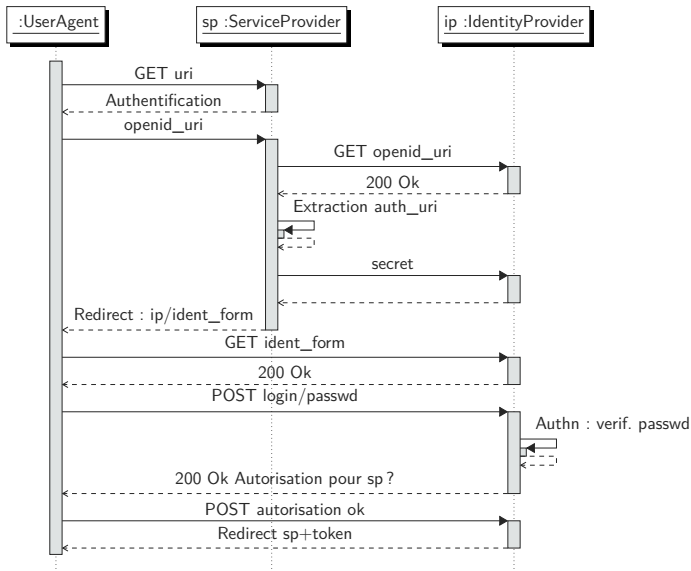
tiers de confiance

<http://openid.net/>, RFC 5849, RFC 6749

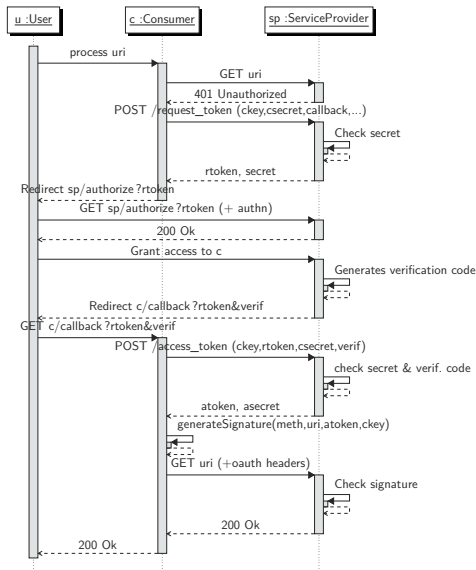
Exemple

- ▶ google
- ▶ facebook
- ▶ twitter
- ▶ stackexchange
- ▶ github
- ▶ ...

OpenID



OAuth



- ▶ authentification ad-hoc
- ▶ spécifique à une application

- ▶ **X** don't reinvent the wheel
- ▶ **X** don't repeat yourself
- ▶ **X** intégration, portail, SSO
- ▶ **X** sécurité
- ▶ ...

legacy

mot de passe

- salé
- haché

— jamais en clair

haché (fonction de hachage crypto)

- ▶ « unique »
- ▶ non réversible
- ▶  MD5
- ▶  SHA-1
- ▶  SHA-256

Exemple

« pas de panique »

► SHA1 : `be38f00728a47a1c205f6b06946339085b026722`

► SHA256 :

`e211cdd14efdeef401940873725896a9aef63fbb5fcb19600bdcbac11807`

 attaque par dictionnaire / rainbow table

sel

- ▶ chaîne quelconque
- ▶ appli ou utilisateur
- ▶ stockée à part
- ▶ ajoutée au mot de passe avant hachage

Exemple (SHA-1)

- ▶ « pas de panique » →
be38f00728a47a1c205f6b06946339085b026722
- ▶ + « \$marvin ! »
- ▶ « pas de panique\$marvin ! » →
3e525b6df88d2eaae52af08ea435134663507368

⇒ bcrypt

Vérification

Exemple

- ▶ user : `zaphod`, password : `pas de panique`
(HTTP Basic, formulaire HTML, ...)
- ▶ `h = select passHash from users where login = 'zaphod'`
- ▶ `assert h == sha1("pas de panique$marvin")!`

Sécurisation des communications — TLS

- ▶ SSL : Secure Socket Layer (RFC 6101)
- ▶ TLS : Transport Layer Security (RFC 5246)
- ▶ https : (RFC 7230, RFC 7231)

entre TCP et protocole applicatif (HTTP)

HTTP

RFC 2817, RFC 2818

- ▶ 101 Switching Protocols
- ▶ 426 Upgrade Required
- ▶ Upgrade

forcer le chiffrement

```
GET / HTTP/1.1  
Host : www.example.com  
Upgrade : TLS/1.2  
Connection : upgrade
```

```
HTTP/1.1 101 Switching Protocols  
Upgrade : TLS/1.2, HTTP/1.1  
Connection : upgrade
```

chiffrement asymétrique

- ▶ authentification du serveur
- ▶ chiffrement des communications
- ▶ intégrité des données
- ▶ authentification du client

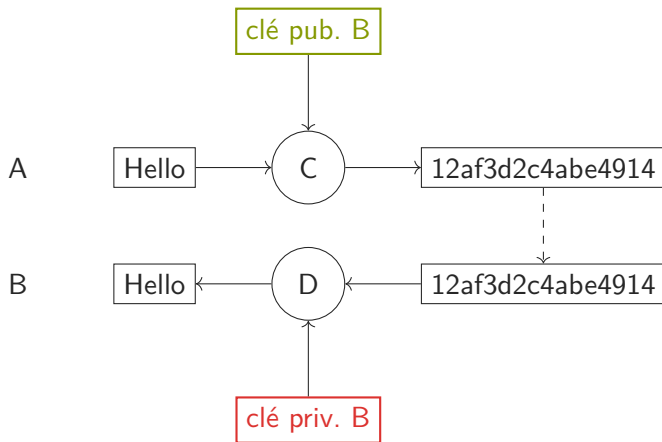
Principe

2 clés :

- ▶ clé publique : peut être divulguée
- ▶ clé privée : gardée secrète (mot de passe)

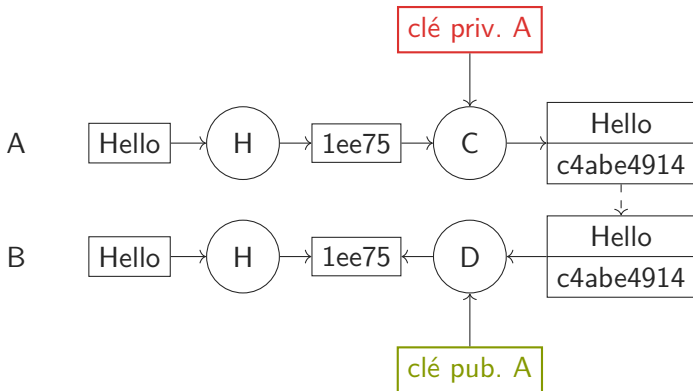
Chiffrement

garantir le destinataire

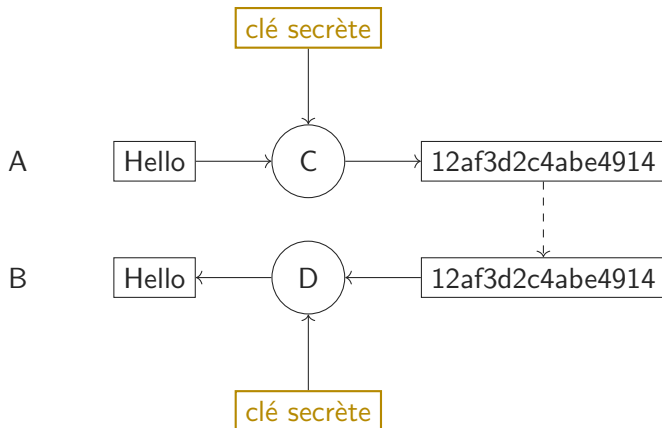


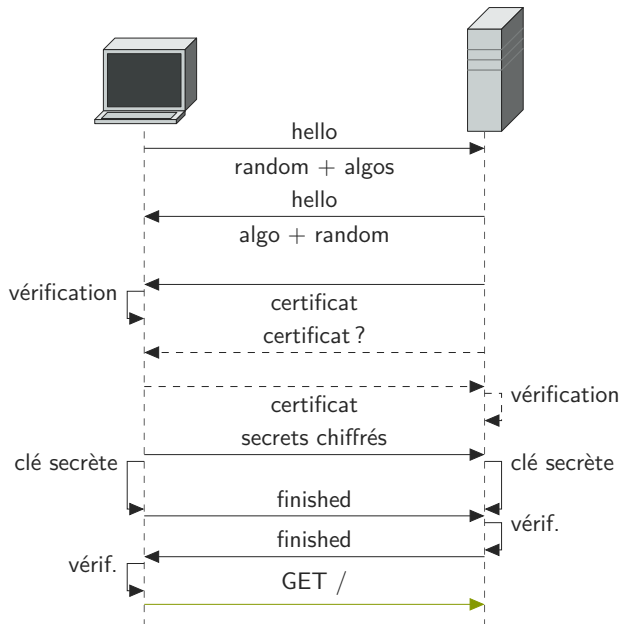
Signature

garantir l'expéditeur / contenu



Chiffrement symétrique





certificat : X.509

- clé publique
- + méta-données (host, utilisation)
- signé par CA (certification authority)

→ PKI (Public Key Infrastructure)

- ▶ ≈ certificats auto-signés
- ▶ ≈ intermédiaires : **CONNECT**
- ▶ ≈ virtual host
- ▶ ≈ mixed content

Attaques sur les applications Web

- ▶ grande surface d'attaque
- ▶ publique
- ▶ ≠ technologies
- ▶ nombreuses couches (client, serveur, réseau, BD)
- ▶ ⇒ nombreuses failles potentielles
- ▶ ⇒ nombreuses attaques possibles

OWASP (Open Web Application Security Project)

<https://cheatsheetseries.owasp.org/>

- ▶ réseau (DDoS)
- ▶ serveur appli
- ▶ machine (OS)
- ▶ langage

envoi de données malicieuses

- ▶ SQL
- ▶ chemin de fichiers
- ▶ code exécuté (shell, `eval`)

force à exécuter du JS

- ▶ stocké
- ▶ soumis

force à exécuter une requête

- ▶ image
- ▶ formulaire
- ▶ click hijacking

similaire XSS

⇒ same-origin policy

JSONp

CORS : Cross-origin resource sharing

- ▶ Origin
- ▶ Access-Control-Allow-Origin
- ▶ Access-Control-Allow-Methods

Test d'applications web

Tests

- ▶ unitaires
- ▶ intégration
- ▶ fonctionnels
- ▶ IHM
- ▶ performances

client-serveur (client riche)

- ▶ tests du serveur
- ▶ tests du client
- ▶ tests d'intégration

⇒ découplage

 ne tester que ce dont on est responsable

- ▶ pas le navigateur
- ▶ pas le serveur d'application
- ▶ pas le framework MVC
- ▶ pas l'ORM
- ▶ ...

inversion de contrôle / injection $\Rightarrow$ mock / stubs

Mock / Stubs

- ▶ « fausse » classe
- ▶ bonne interface
- ▶ comportement connu
- ▶ *injectée*
- ▶ remplace les dépendances
- ▶ classe testée isolée

modèle $\Rightarrow$ règles « métier »

sans l'ORM/DAO

sans la V et C (directement méthodes)

contrôleur $\Rightarrow$ routage

sans HTTP

sans la V et le M

selon la requête :

- ▶ bon modèle
- ▶ bonne opération
- ▶ bonne vue
- ▶ ...

création objet requête « manuellement »

vue $\Rightarrow$ représentation
génération et comparaison

⇒ profil « test » (configuration framework)

→ choix du framework

- ▶ application complète (MVC)
- ▶ sans serveur d'application
- ▶ sans base de données

- ▶ application non déployée
- ▶ BD de test en mémoire
- ▶ appel direct du contrôleur (objet requête)

Point de vue HTTP / API (pas IHM)

⇒ logique du service

- ▶ bonnes ressources
- ▶ codes de retour
- ▶ négociation, requêtes conditionnelles
- ▶ flot de navigation
- ▶ authentification / autorisation

- ⇒ design pour
- ⇒ indépendance par rapport au client
- ⇒ découplage (REST)
- ⇒ données de test

⇒ client automatique

- ▶ simuler comportement utilisateur
- ▶ vérifier le résultat

outils d'automatisation

- ▶ requêtes (url)
- ▶ soumission de formulaire
- ▶ gestion des cookies
- ▶ vérification code retour
- ▶ vérification contenu

⇒ workflow



Javascript

Test de déploiement

- ▶ mise à jour
- ▶ migration BD

- ▶ code client (JS)
- ▶ validité (HTML, CSS)
- ▶ accessibilité (WAI)



différents navigateurs

tests javascript

 frameworks MVC client

⇒ outils de tests JS

⇒ mock (du serveur)

⇒ design decouplé (hypermédia)

automatisation : enregistrer session utilisateur + rejouer



design d'interface

- ▶ capture d'image de référence
- ▶ comparaison



« responsive »

performances / stress

- ▶ passage à l'échelle
- ▶ infrastructure

- ▶ automatisation client
- ▶ exécution parallèle

mesures :

- ▶ temps de réponse (latence)
- ▶ nombre de requêtes concurrentes
- ▶ reprise sur panne (cloud)
- ▶ « stress », « fuzzy »

performances client

- ▶ latence perçue
- ▶ performances JS

Automatisation client

- ▶ twill
- ▶ mechanize
- ▶ webinject
- ▶ selenium
- ▶ capybara

performances/stress : httpperf, bombardier, gatling

debug/intégration

- ▶ curl / httpie : client HTTP
- ▶ mitmdump : proxy

outils navigateur

- ▶ debug JS
- ▶ analyse trafic
- ▶ performances
- ▶ ...

frameworks

- ▶ HttpUnit
- ▶ HtmlUnit : JS
- ▶ JWebUnit : JUnit + HtmlUnit + Selenium

- ▶ application serveur
- ▶ réseau
- ▶ client (JS)

Définition (Latence)

- ▶ temps total : déclenchement → fin affichage
- ▶ temps perçu

→ asynchrone

- ▶ 2ms → non corrélation
- ▶ 5s–10s → perte utilisateur
- ▶ 10% de perte par sec. supplémentaire

↘ temps :

- ▶ connexion
- ▶ génération réponse
- ▶ transfert
- ▶ affichage / exécution JS

- ▶ client
- ▶ application serveur
- ▶ http / infrastructure

- ▶ requête DNS
- ▶ connexion TCP
- ▶ envoie requête
- ▶ réception réponse

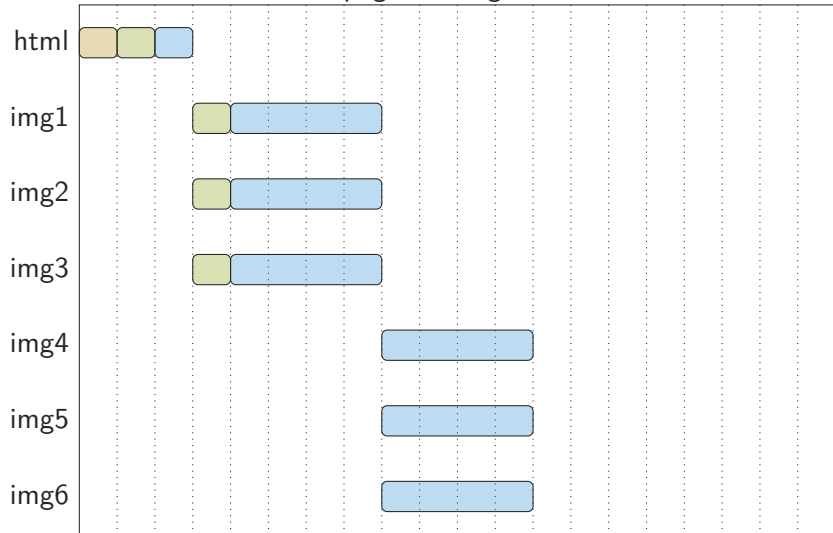
- ▶ 1 pour le HTML
- ▶ 1 par sous-ressource (images, CSS, JS)

navigateurs : 6–8 requêtes parallèles (par domaine)

$$\textit{temps} > \frac{\textit{taille}}{\textit{bande passante}}$$

influence

- design
- structure de la page

1 page, n images

Domain sharding

- ▶ $\neq$ serveurs (nom) `static1.example.com`, `static2.example.com`, `static3.example.com`...
- ▶ $\Rightarrow$ CDN

- ✗ + de temps de connexion
- ✓ + de connexions parallèles

Inlining

- ▶ sprites CSS : `background-image background-position`
- ▶ fusion script et feuilles de styles
- ▶ images :
`data:image/png;base64,iVBORw0KGgoAAAANSUhE...`
(CSS)

Inlining

- ✗ redondance
- ✗ gestion
- ✗ taille
- ✗ chargement à la demande
- ✗ transferts parallèles
- ✗ cache

Inlining

- ✓ petits fichiers (< 3–5 ko)
- ✓ fichiers peu inclus

⇒ équilibre : moins de requêtes ↔ transferts parallèles

Gestion du cache

éviter les requêtes inutiles

- ▶ URL canoniques
- ▶ Expires
- ▶ Cache-Control
- ▶ requêtes conditionnelles (date de modif.)

 généré : défaut pas de cache

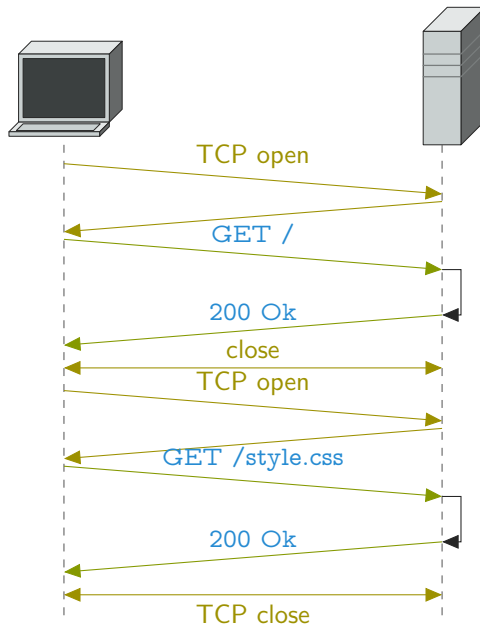
⇒ ressources externes

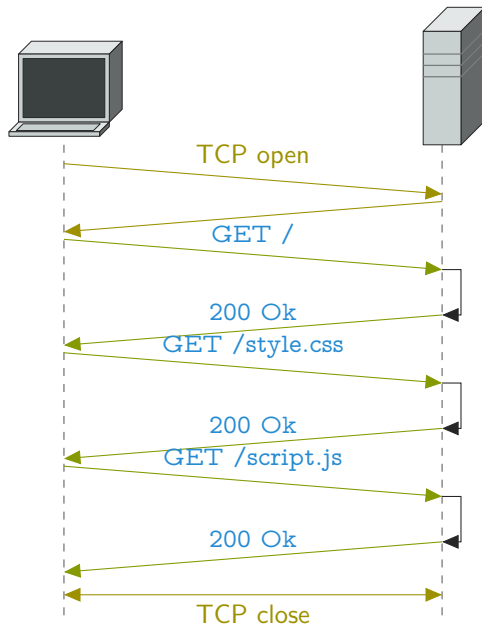
⇒ conception appli/données

Réseau

Connection : keep-alive réutilisation connexion

- ▶ temps de connexion
- ▶ temps de « chauffe » TCP





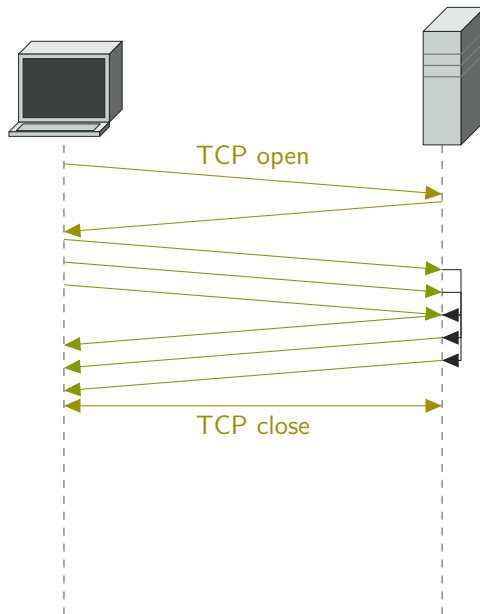
Réseau

pipelining HTTP

- ▶  réponses → même ordre que requêtes
- ▶  requêtes idempotentes
- ▶  forte latence
- ▶  requêtes bloquantes → ordre
- ▶  échec

clients : désactivé par défaut

⇒ HTTP/2.0 : multiplexage



requêtes DNS

- ▶ ✗ temps de résolution
- ▶ ✗ URL avec IP
- ▶ ⇒ URL relatives ou canoniques
- ▶ ✓ cache DNS

Redirections

- ▶ corps dans réponse (POST)
- ▶ style *responsive* plutôt que redirection `mobile.example.com`
- ▶ `/` en fin d'URL (statique)

GET /articles HTTP/1.1

Host : www.example.com

HTTP/1.1 301 Moved Permanently

Location : http://www.example.com/articles/

GET /articles/ HTTP/1.1

Host : www.example.com

HTTP/1.1 200 OK

Content-Location : index.html

Content-Type : text/html; charset=UTF-8

Compression

- ▶ **Accept-Encoding / Content-Encoding** → gzip
 - ▶ « gros contenus »
 - ▶ contenu texte
 - ▶ à la demande → temps CPU
 - ▶ *a priori* → espace de stockage
 - ▶ → temps CPU client
- ▶ images
- ▶ code redondant
 - ▶ JS : abstraction, HOF
 - ▶ CSS : sélecteurs
- ▶ minimisation

Données

- ▶ cookies (domain/path)
- ▶ formulaires (nom de champs)
- ▶ màj incrémentale (ajax) $\Rightarrow$ + de requêtes

✗ JS bloque

⇒ fin de page

`<script async defer src="">`

- ▶ $\emptyset$ → téléchargé + exécuté, continue
- ▶ `defer` téléchargé, continue, exécuté
- ▶ `async` téléchargé + exécuté + continue

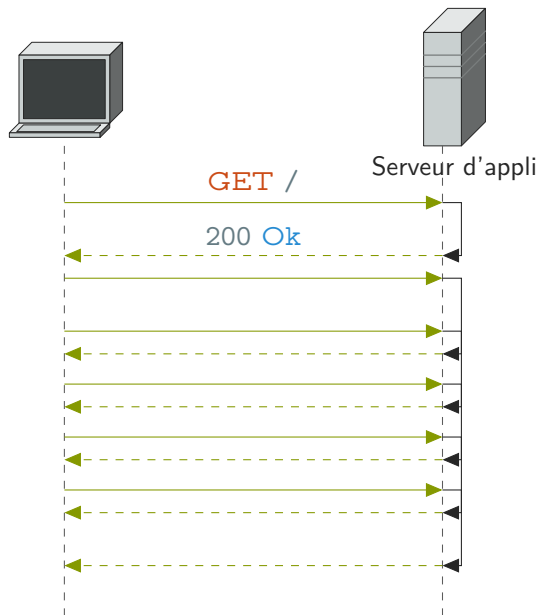
- ▶ <https://developers.google.com/speed/>
- ▶ <http://developer.yahoo.com/performance/rules.html>
- ▶ <https://developer.mozilla.org/en-US/docs/Web/Performance>

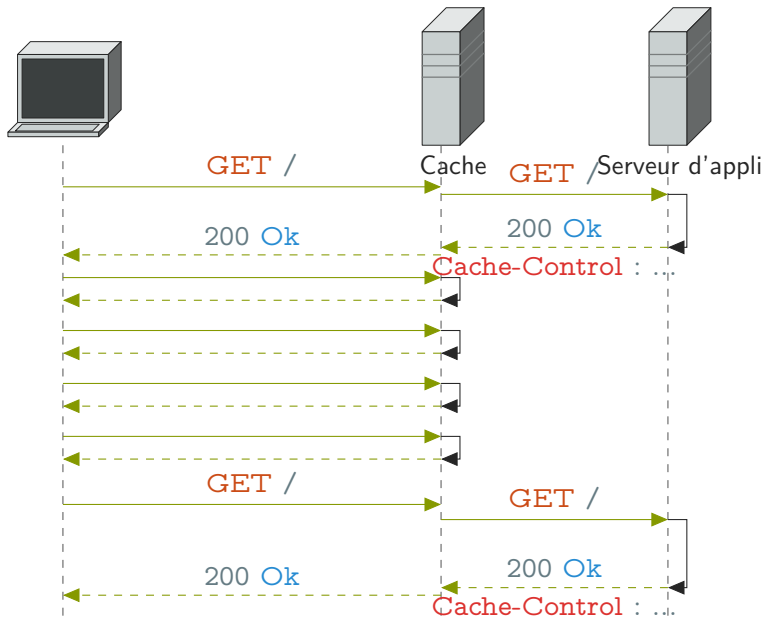
augmenter les capacités de l'application

- ▶ + utilisateurs (concurrents)
- ▶ + données
- ▶ + de traitements

frontal en *reverse proxy*

Varnish





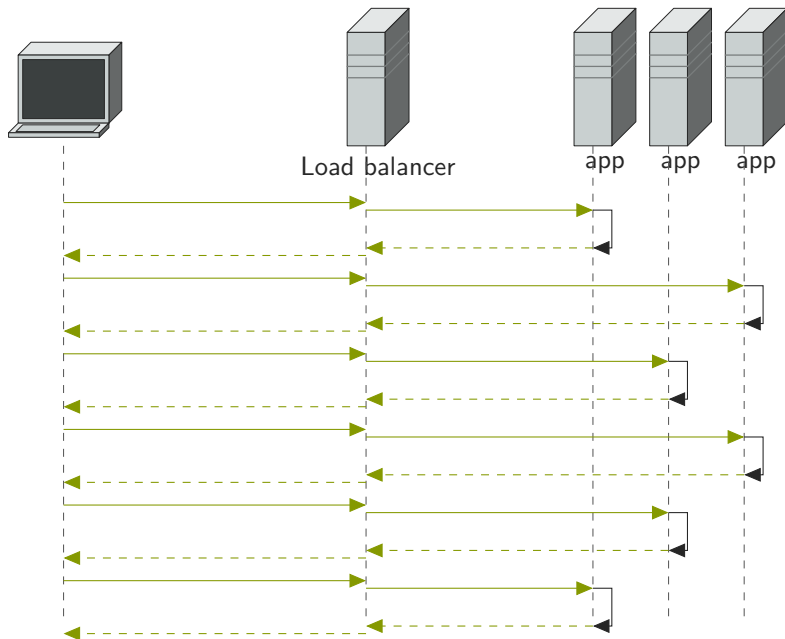
cachable

- ▶ sans effet de bord
- ▶ sans état



sessions

- ▶ distribution des requêtes
- ▶ réplication de l'application
- ▶ séparation des fonctionnalités



- ▶ DNS : 1 nom $\mapsto n$ IP *publiques*
- ▶ routeur : 1 IP publique $\mapsto n$ IP internes
- ▶ reverse proxy : niveau HTTP (HAProxy)

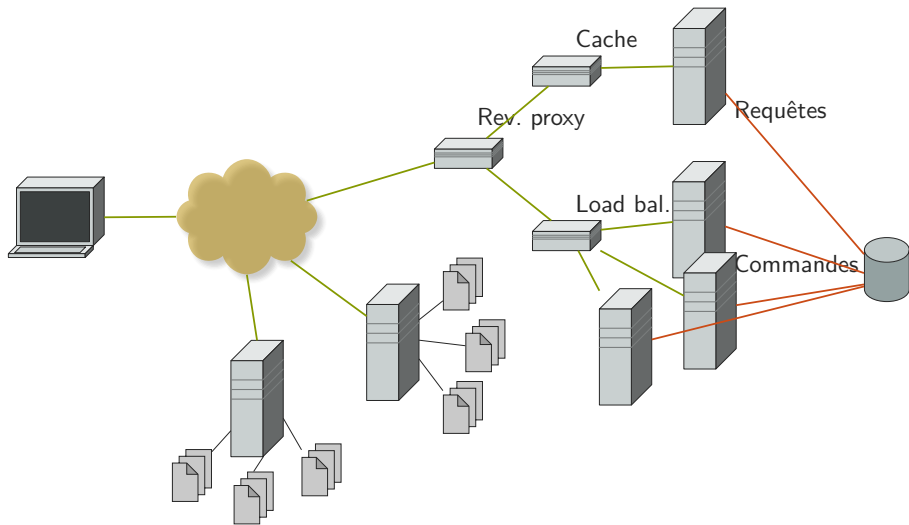


sessions

- ▶ LB → ID de session
- ▶ réplication des sessions

- ▶ 2 sous-applications
- ▶ lecture : cache
- ▶ modification : répartition

⇒ micro-services



- ▶ données distribuées
- ▶ données répliquées
- ▶ données en RAM
- ▶ cache partagé (memcached, redis)
- ▶ accès asynchrone

- ▶ IO
- ▶ → toute la chaîne
- ▶ *callback* (continuation passing)
- ▶ événementiel non bloquant

Node.js, GUnicorn, . . .

- ▶ monitoring
- ▶ orchestration
- ▶ provisioning, gestion de conf.
- ▶ VM / conteneur

ZooKeeper, monit, nagios

Chef, Puppet, Ansible

Vagrant, Docker

- ▶ <http://www.lavajug.org/2015/05/21/applications-concurrentes-polyglottes>
- ▶ <http://www.lavajug.org/2015/09/23/the-gradle-way>
- ▶ <http://www.lavajug.org/2015/10/15/docker>
- ▶ <http://www.lavajug.org/2015/12/17/gatling>
- ▶ <http://www.lavajug.org/2016/09/29/github-ci>
- ▶ <http://www.lavajug.org/2016/05/25/event-sourcing>

Analyse de trafic

mesurer l'utilisation

changements $\Rightarrow$ conséquences

→ choix (design)

- ▶ ancienne fonctionnalité → moins utilisée
- ▶ nouvelle fonctionnalité → pas utilisée

→ bugs

- ▶ problème de déploiement

amélioration

- ▶ séquence d'opération $\Rightarrow$ nouvelles fonctionnalités
- ▶ navigabilité
- ▶ performances
- ▶ suppression de fonctionnalités

marketing

- ▶ publicité
- ▶ indexation
- ▶ visibilité

⇒ retour sur l'utilisation

Bounced

- ▶ départ immédiat
- ▶ pas de poursuite

élevé $\Rightarrow$ problème

Landing page

page d'arrivée sur le site

- ▶ accueil
- ▶ lien profond

⇒ topologie du site (pas URL)

cf. Google vs. presse

Conversion

but final

- ▶ achat
- ▶ inscription
- ▶ téléchargement
- ▶ ...

Funnel

suite d'action : arrivée → conversion

→ 3 clicks

Medium

ou source

→ comment l'utilisateur est arrivé

- ▶ autre page ([Referer](#))
- ▶ saisie directe
- ▶ moteur de recherche
- ▶ mailing
- ▶ flux
- ▶ réseaux sociaux
- ▶ campagne de com. (QRCode)
- ▶ ...

Session

- ▶ temps passé sur le site
- ▶ difficile à mesurer
- ▶ 30 min → expiration

mesure *on site* $\Rightarrow$ utilisateurs effectifs

- ▶ statistiques de visites
- ▶ comportement
- ▶ performances

mesure *off site* : environnement $\Rightarrow$ utilisateurs potentiels

- ▶ commentaires
- ▶ réseaux sociaux
- ▶ moteurs de recherche

$\rightarrow$ buzz

mesures de 1^{er} niveau → « directes »

- ▶ nb. visites (/j)
- ▶ nb. visiteurs (/j)
- ▶ nb. visites / page
- ▶ temps moyen (→ session)
- ▶ temps moyen / page
- ▶ taux / fréquence de retour
- ▶ profondeur visitée (arrivée, source)
- ▶ répartition lieu / langue
- ▶ ...

mesures de 2^e niveau → inférées

- ▶ ≠ comportement utilisateur ↔ visiteur*
- ▶ ≠ comportement selon la source
- ▶ *bounce* par page
- ▶ *bounce* par source
- ▶ recherche interne
- ▶ temps / nb. de visite ⇒ conversion
- ▶ ...

- ▶ log : serveur
- ▶ *page tag* : client



informations $\neq \rightarrow$ ne pas comparer

approches complémentaires

- ▶ caches et proxy :  logs
- ▶ événements clients :  logs
(fermeture, etc.)
- ▶ cookies (tiers) :  scripts
- ▶ clients sans JS :  logs
- ▶ robots :  logs
- ▶ impact client (perfs) :  logs
- ▶ blocage :  scripts
- ▶ identification :  logs

- ▶ simple
- ▶ sûr
- ▶  caches

source

⇒ [Referer](#)

- ▶ ✓ sites tiers (contexte)
- ▶ ✓ mots-clés de recherche
- ▶ ✗ redirection, raccourcisseurs
- ▶ ✗ source privée (webmail, réseau social)
- ▶ ✗ *single page* → fragments

⇒ URI spécifique ([GET](#)) (query string)

[?utm_source=RSS&utm_medium=news&utm_campaign=example](#)

- ▶ mailing, flux RSS, QRCode
- ▶ ✗ stockage
- ▶ ✗ partage

identification

⇒ IP → connexion

- ▶  IP dynamique (mobile) : 1 utilisateur → ≠ IP
- ▶  proxy, hotspot : ≠ IP → 1 utilisateurs

⇒ cookie → navigateur

- ▶  pas de session → identification
- ▶  changement IP
- ▶  proxy
- ▶  supprimé/bloqué
- ▶  caches

⇒ identification client

- ▶  poste partagé : 1 client → ≠ utilisateurs
- ▶  terminaux différents : ≠ clients → 1 utilisateur

 corrélation des requêtes

 métriques utilisateur

⇒ statistiques, pas sécurité

robots

- ▶  stats spécifiques
- ▶  biais si pas identifié

⇒ filtrage

- ▶ **User-Agent** → liste complète
⇒ faux négatif
- ▶ `/robots.txt`
⇒ faux positif
- ▶ comportement...



cache

- requêtes non vues
- cache partagé
- cache frontal, load balancing $\Rightarrow$ stat sur le reverse proxy

$\Rightarrow$ [no-cache](#) $\rightarrow$ perfs

$\Rightarrow$ revalidation

$\Rightarrow$ image tracking : [Referer](#) $\rightarrow$ cible



perte de la source

Outils

- ▶ analog
- ▶ webalizer
- ▶ visitors
- ▶ piwik
- ▶ logstash + kibana

→ événements clients

- ▶ interruption
- ▶ fermeture
- ▶ quitte le site
- ▶ défilement
- ▶ déplacement de la souris
- ▶ ...

⇒ dépendant config. client

- ▶ désactivé
- ▶ bloqué
- ▶ cookies (tiers)

→ 1-5%

- ▶ exécution script client
- ▶ interception événements
- ▶ cookies (corrélation)

javascript

- ▶ ≈ balise sur chaque page → oublis
- ▶ ✗ erreurs autres scripts → blocage
- ▶ ✗ mauvais handler (`addEventListener`)
- ▶ ≈ performances : début / fin
bas → haut : +20% mesuré (Google)

cookies tiers

- ▶ bloqués
- ▶ supprimés → taux de retour
+ délais long